

LSL

Serveur web

par Bestmomo Lagan

Version 1.5

30/08/09

Avertissement : Ce guide n'est pas un document officiel de Second Life. Il n'engage que la responsabilité de son auteur.

Table des Matières

1. Introduction.....	4
2. Les outils.....	5
2.1 Serveur.....	5
2.2 Client FTP.....	5
2.3 phpMyAdmin.....	5
2.4 Editeur PHP.....	5
3. MySQL.....	6
3.1 Notion de base de donnée relationnelle.....	6
3.2 phpMyAdmin.....	7
3.3 Requêtes SQL.....	14
3.3.1 SELECT.....	14
3.3.2 INSERT.....	18
3.3.3 DELETE.....	18
3.3.4 UPDATE.....	19
3.4 Pour aller plus loin.....	20
4. PHP.....	21
4.1 EasyPhp.....	21
4.2 Les bases.....	22
4.3 Les variables.....	22
4.4 Les tableaux.....	24
4.4.1 Tableaux indicés.....	24
4.4.2 Tableaux associatifs.....	24
4.4.3 Fonctions pour les tableaux.....	25
4.5 Chaînes de caractères.....	27
4.6 Les fonctions.....	29
4.7 Les structures de contrôle.....	31
4.8 Les objets.....	31
4.8.1 Version PHP4.....	31
4.8.2 Version PHP5.....	32
4.9 FTP.....	33
4.10 Pour aller plus loin.....	33
5. Communication.....	35
5.1 Liaisons entre scripts de PHP.....	35
5.2 Liaisons entre PHP et MySql.....	35
5.2.1 Version PHP4.....	36
5.2.2 Version PHP5.....	37
5.2.3 Protocole HTTP.....	37
5.2.3.1 Requête du client.....	38
5.2.3.2 Reponse du serveur.....	38
5.2.3.3 Traitement par PHP.....	39
5.2.3.4 Traitement par LSL.....	40
6. Synthèse.....	42
6.1 Le cahier des charges.....	42
6.2 La base de données.....	42
6.3 Le code PHP.....	44

6.3.1	Routines.....	44
6.3.1.1	constantes.inc.php	44
6.3.1.2	connexion.inc.php	44
6.3.1.3	requete.inc.php	44
6.3.1.4	infos.inc.php.....	45
6.3.2	initialisation.php.....	45
6.3.3	vote.php.....	47
6.3.4	classement.php.....	48
6.3.5	reset.php.....	49
6.4	Le code LSL.....	49
6.4.1	Script de vote.....	49
6.4.2	Script de la centrale.....	51
7.	Serveur LSL.....	55
7.1	Une URL temporaire.....	55
7.2	URL disponibles.....	56
7.2.1	Limitations liées aux régions.....	56
7.2.2	Limitations liées aux résidents.....	56
7.2.3	Limitations des requêtes.....	56
7.3	Réponses aux requêtes.....	57
7.4	Un exemple simple.....	57

1. Introduction

LSL est un langage de script puissant mais qui souffre de quelques limitations souvent gênantes. En particulier il n'y a aucune persistance des données. Les variables ne conservent leur valeur que pendant la durée de fonctionnement du script, que survienne un **reset** et tout est perdu. D'autre part il n'y a aucun système de gestion de données digne de ce nom et il faut jongler avec des **notecards** qui ne fonctionnent évidemment qu'en lecture. Un autre problème concerne l'espace mémoire disponible pour un script, même si mono nous offre désormais 64K au lieu des 16K dont nous disposions précédemment c'est une limite infranchissable et il faut multiplier les scripts pour les applications gourmandes.

Il en résulte qu'il est souvent nécessaire de recourir à un serveur externe pour disposer à la fois d'un système de gestion de données efficace et d'une puissance de traitement incomparable. **LSL** autorise la communication avec un serveur externe au moyen du protocole **HTTP** qui est le protocole standard de communication sur la toile. Ce n'est pas forcément élégant et toujours très efficace mais ça permet de communiquer pas mal d'informations. Mais pour utiliser un serveur encore faut-il en disposer d'un. Heureusement il existe de nombreuses offres gratuites avec le couple classique **PHP** et **MySQL**. Il ne faut pas s'attendre à des performances extraordinaires sur ces serveurs mutualisés gratuits mais ils sont largement suffisants pour des utilisations pas trop exigeantes. Si vous pensez avoir besoin d'un débit important il faut vous tourner vers une offre payante en lisant bien les conditions générales parce que l'offre foisonnante cache souvent de mauvaises surprises.

La mise en œuvre d'une communication avec un serveur et la gestion des informations sur celui-ci impliquent un certain nombre d'outils et de connaissances qui ne sont pas forcément répandues parmi les scripteurs de **LSL**. C'est ce qui explique que cette possibilité soit assez peu utilisée et suscite de nombreuses questions sur les forums sans que les réponses apportent une aide totale tant le sujet est vaste. D'autre part il n'y a pas de tutorial vraiment complet sur la toile concernant ce sujet. Je me suis dit qu'un guide sur le sujet serait forcément bien accueilli mais j'ai longtemps hésité devant l'ampleur de la tâche et aussi le manque de disponibilité. D'autre part je ne suis pas un habitué de **PHP** (dois-je avouer mes préférences pour un certain éditeur ?...). J'ai été amené à développer certains sites dynamiques avec cette technologie mais je n'ai jamais ressenti d'affinité particulière pour ce langage et la sobriété des outils disponibles.

Il y avait plusieurs façons d'aborder ce sujet, j'ai choisi comme d'habitude la simplicité quitte à occulter des éléments importants. La démarche didactique implique des choix parfois difficiles. Je vous laisse juger de ceux-ci. Considérez ce document comme une modeste introduction, chacun de ses chapitres mériterait à lui seul un ouvrage complet.

Cette première version est appelée à s'améliorer, elle ne traite que de requêtes dans le sens client -> serveur. Il y aurait aussi beaucoup à dire concernant le **XML-RPC**, qui permet les requêtes serveur -> client, ne serait-ce qu'une présentation même sommaire du **XML**.

Et évidemment faites moi part des bugs que vous ne manquerez pas de rencontrer au fil de ces pages !

2. Les outils

Un bon artisan possède de bons outils. Que nous faut-il dans notre cas ? Il nous faut un serveur, un outil de gestion de base de données sous **MySQL**, un outil d'édition de scripts **PHP**, un outil de transmission de fichier sous **FTP**. Nous allons détailler tout ça.

2.1 Serveur

Qui dit serveur dit ordinateur et qui dit ordinateur dit système d'exploitation. Vous savez qu'aujourd'hui deux systèmes d'exploitation concurrents dominent le marché : la version payante avec **Windows** et **SQL Serveur** et la version Open Source avec **Linux** et **Apache**. Par définition l'option payante est plus chère que la gratuite malgré les tentatives de démonstration alambiquées de certains. Un serveur permet de stocker des fichiers, des données, de créer des scripts. Il existe des serveurs dédiés et des serveurs mutualisés. Dans le premier cas le serveur ne sert que pour vous, dans le second cas vous le partagez avec d'autres utilisateurs. Vous vous doutez également qu'un serveur dédié vous coûtera plus cher qu'un serveur mutualisé mais que les performances seront différentes. Pour vos premiers pas dans le monde des serveurs je vous conseille une offre gratuite du genre de celle de **WebDynamit**, **Freesurf** ou **Free**.

Vous pouvez également installer un serveur local sur votre ordinateur pour faire des tests. Il existe plusieurs programmes « tout en un » mais il me semble que **EasyPHP** est le plus pratique. Il permet l'installation simplifiée d'un serveur **Apache**, de **MySQL** et de **PHP**. Vous pouvez le télécharger à cette adresse :

<http://www.easyphp.org>

Les prochains chapitres nécessitent la présence de ces applications pour réaliser les exemples proposés.

2.2 Client FTP

FTP signifie « File Transfert Protocol ». C'est un protocole de transmissions de fichiers par Internet fonctionnant selon le modèle client-serveur. Le client permet d'envoyer les fichiers sur le serveur **FTP**. Le client le plus connu est **FileZilla**. Il est gratuit, simple et complet. Une autre option que j'utilise personnellement est **Total Commander**. C'est une application versatile qui regroupe entre autres : explorateur de fichier avec reconnaissance des compressions les plus répandues et client **FTP**.

2.3 phpMyAdmin

Pour gérer une base de données **MySQL** vous avez besoin d'un outil d'administration. Pratiquement tous les hébergeurs proposent **phpMyAdmin**. Il s'agit d'une application puissante, simple et efficace. Vous l'avez aussi en local avec **EasyPHP**. C'est celle que j'utiliserai dans ce document.

2.4 Editeur PHP

Pour éditer vos scripts **PHP** vous avez plusieurs options gratuites, en Open Source **NetBeans** et **PHPEclipse**. En gratuit **PHPCoder**.

3. MySQL

3.1 Notion de base de donnée relationnelle

MySQL est un SGBDR, un Système de Gestion de Bases de Données Relationnelles. Explicitons un peu cela.

Il est difficile de dire exactement ce qu'est une « base de données », disons simplement que c'est une façon de stocker et d'organiser des informations de façon à pouvoir les gérer. Il existe de nombreuses situations où des informations sont à conserver et organiser. Imaginez par exemple votre bibliothèque personnelle contenant des centaines d'ouvrages. Vous désirez savoir facilement quels sont les livres que vous possédez sans être obligé de les regarder un par un, comment allez-vous procéder ? Vous pouvez par exemple noter les titres et les auteurs dans une liste et ensuite classer par ordre alphabétique les titres et une autre liste avec les auteurs classés également par ordre alphabétique. Vous pouvez compléter ces listes en notant l'emplacement exact de l'ouvrage : étagère et case par exemple. Vous avez établi ainsi une base de données manuelle. Il vous est maintenant très facile de retrouver un ouvrage par titre ou par auteur. Chaque fois que vous acquérez un nouveau livre vous l'intégrez dans les listes. Il en est de même lorsque vous en supprimez ou prêtez un. Mais vous pouvez vous rendre compte rapidement qu'une gestion manuelle n'est pas vraiment pratique et penser à une gestion informatisée. Schématisons vos données . Voici votre liste classée par auteurs :

Auteur	Titre	Emplacement
Flaubert Gustave	Madame Bovary	C1
Flaubert Gustave	Mémoires d'un fou	C2
Hugo Victor	L'homme qui rit	A0
Hugo Victor	Les misérables	A1
Hugo Victor	Notre-Dame de Paris	A2

Voici la même liste classée par titres :

Auteur	Titre	Emplacement
Hugo Victor	L'homme qui rit	A0
Hugo Victor	Les misérables	A1
Flaubert Gustave	Madame Bovary	C1
Flaubert Gustave	Mémoires d'un fou	C2
Hugo Victor	Notre-Dame de Paris	A2

L'informatique permet facilement de trier ce tableau comme je viens de le faire dans **Word** pour ce document. Mais il serait plus pratique d'avoir un programme dédié qui s'occupe de ces données, les conserve, et permette de les gérer facilement. C'est justement le but des **SGBD**. En terminologie base de donnée le tableau ci-dessus s'appelle **table**, chaque colonne se nomme **champ** et chaque ligne **enregistrement**. La table que nous avons constituée possède donc 3 champs : « Auteur », « Titre » et « Emplacement ». Elle a aussi 5 enregistrements en mémoire. Si nous observons notre table nous pouvons constater une chose : des noms d'auteurs sont répétés, ce qui n'est pas très judicieux, si je dois corriger l'orthographe d'un nom je dois le faire sur chaque enregistrement où il apparaît. D'autre part l'occupation mémoire n'est pas optimisée.

Pour remédier à ces imperfections on peut diviser nos données en deux tables : une pour les auteurs et une autre pour les ouvrages. La table pour les auteurs devient ceci :

Nom
Flaubert Gustave
Hugo Victor

Cette fois chaque nom d'auteur n'apparaît qu'une fois. Et voici la table pour les ouvrages :

Titre	Emplacement
L'homme qui rit	A0
Les misérables	A1
Madame Bovary	C1
Mémoires d'un fou	C2
Notre-Dame de Paris	A2

Maintenant nous avons optimisé nos données mais survient un nouveau problème, on n'a plus de relation entre les auteurs et les ouvrages. Une façon de faire consiste à indexer la table des auteurs :

ID_Auteur	Nom
1	Flaubert Gustave
2	Hugo Victor

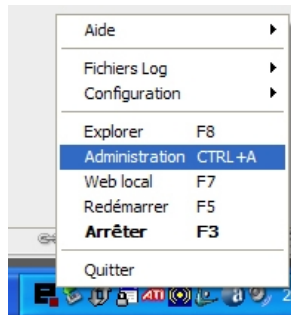
Et à reporter cet index dans la table des ouvrages :

Titre	Emplacement	ID_Auteur
L'homme qui rit	A0	1
Les misérables	A1	1
Madame Bovary	C1	2
Mémoires d'un fou	C2	2
Notre-Dame de Paris	A2	1

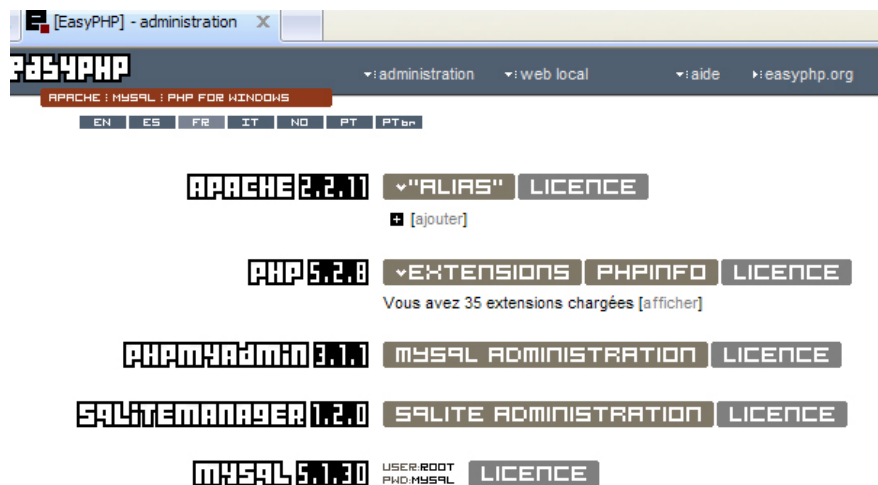
Maintenant nos deux tables sont reliées. Vous voyez apparaître la notion de relation entre tables. Le champ **ID_Auteur** qui apparaît dans la table des ouvrages est nommé **clef étrangère**, parce qu'elle n'est pas une donnée de la table mais sert uniquement de lien. Le champ **ID_Auteur** dans la table des auteurs est nommé **clef primaire**. Maintenant que les bases sont posées voyons un peu comment nous pouvons entrer tout ça dans une base MySQL...

3.2 phpMyAdmin

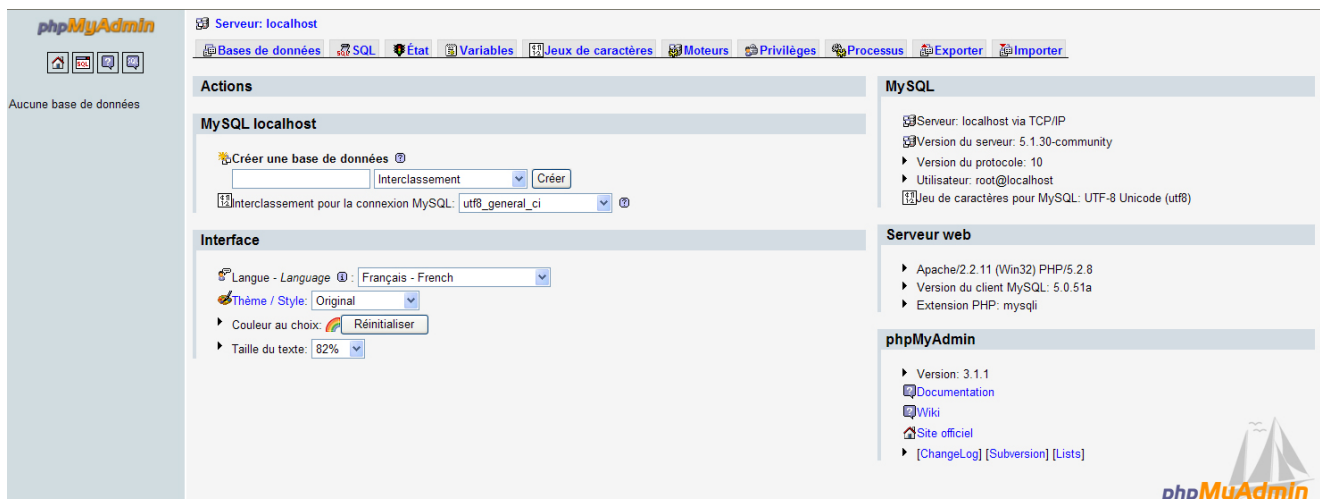
Si vous avez installé **EasyPHP** sur votre ordinateur vous avez accès au gestionnaire **phpMyAdmin**. Pour cela commencez par lancer **EasyPHP**, vous obtenez une petite icône dans votre zone de lancement rapide. Faites un clic droit dessus et choisissez « Administration » :



Vous obtenez ceci :

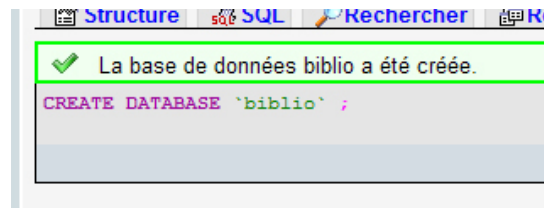


Cliquez sur « MySQL Administration ». Vous arrivez ici :



Pour le moment nous n'avons aucune base de donnée. Nous allons en créer une pour nos tests :

Appeler la « biblio » et cliquez sur « Créer ».



Un message rassurant apparaît au-dessus d'une zone qui va nous intéresser. Vous pouvez lire :

CREATE DATABASE `biblio` ;

Pour parler à **MySQL** vous avez besoin d'un certain langage, il s'agit de **SQL** (Structured Query Language). Ici **phpMyAdmin** vous a gentiment écrit la bonne syntaxe pour créer une base nommée « biblio ». Pour peu que l'anglais ne vous soit pas trop étranger vous comprenez facilement ce qui est écrit là. Maintenant **phpMyAdmin** affiche qu'il y a une table :



Il nous dit aussi qu'il n'y a aucune table dans cette base. Cliquez sur le nom de la base, vous obtenez cet écran :

Puisqu'on nous le propose si gentiment nous allons créer la table pour les auteurs :

 **Créer une nouvelle table sur la base biblio**

Nom: Nombre de champs:

Cliquez sur « Exécutez » vous arrivez ici :

Serveur: localhost ▶ Base de données: biblio ▶ Table: Auteurs

Champ		
Type ?	INT	INT
Taille/Valeurs ¹		
Défaut ²	aucune	aucune
Interclassement		
Attributs		
Null	☐	☐
Index	---	---
A_I	☐	☐
Commentaires		

Commentaires sur la table:

Moteur de stockage: ? InnoDB

Interclassement:

Définition de PARTITION: ?

Il ne nous reste plus qu'à saisir les champs pour notre table :

Serveur: localhost ▶ Base de données: biblio ▶ Table: Auteurs

Champ	ID_Auteur	Nom
Type ?	INT	VARCHAR
Taille/Valeurs ¹		100
Défaut ²	aucune	aucune
Interclassement		
Attributs		
Null	☐	☐
Index	PRIMARY	---
A_I	☒	☐
Commentaires		

Le type du champ est une notion importante. Les données à stocker peuvent être des nombres, des chaînes de caractères, des dates... Il nous faut spécifier le type de donnée que nous voulons mémoriser dans le champ. Vous pouvez remarquer en cliquant sur le bouton déroulant qu'il y a de très nombreux types disponibles. Pour notre champ d'index nous allons prendre simplement un type numérique entier de base et pour notre champ « Nom » un type chaîne de caractère d'une longueur maximale 100. Ensuite nous précisons juste que le champ « ID_Auteur » doit être une clef primaire auto incrémentée, ce qui signifie qu'elle commencera à 1 et à chaque enregistrement sera augmentée de 1. Cliquez sur « Sauvegarder », vous arrivez ici :

Serveur: localhost ► Base de données: biblio ► Table: Auteurs

Afficher Structure SQL Rechercher Insérer Exporter Importer Opérations Vider Sup

✓ La table 'biblio'. 'Auteurs' a été créée.

```
CREATE TABLE `biblio`.`Auteurs` (
  `ID_Auteur` INT NOT NULL AUTO_INCREMENT PRIMARY KEY ,
  `Nom` VARCHAR( 100 ) NOT NULL
) ENGINE = INNODB
```

	Champ	Type	Interclassement	Attributs	Null	Défaut	Extra	Action					
☐	ID_Auteur	int(11)			Non	aucune	auto_increment						
☐	Nom	varchar(100)	utf8_general_ci		Non	aucune							

↑ Tout cocher / Tout décocher Pour la sélection :

Version imprimable Gestion des relations Suggérer des optimisations quant à la structure de la table

Ajouter 1 champ(s) ☒ En fin de table ☐ En début de table ☐ Après ID_Auteur Exécuter

Index:

Action	Nom de l'index	Type	Unique	Compressé	Champ	Cardinalité	Interclassement	Null	Commentaire
	PRIMARY	BTREE	Oui	Non	ID_Auteur	0	A		

Cette fois encore **phpMyAdmin** nous a gentiment écrit la syntaxe SQL de notre commande. Créons de la même façon la table « Ouvrages » :

✱ Créer une nouvelle table sur la base **biblio**

Nom: Nombre de champs:

Serveur: localhost ▶ Base de données: biblio ▶ Table: Ouvrages

Champ	Titre	Emplacement	ID-Auteur
Type	VARCHAR	CHAR	INT
Taille/Valeurs ^{*1}	100	2	
Défaut ²	aucune	aucune	aucune
Interclassement			
Attributs			
Null	☐	☐	☐
Index	UNIQUE	---	---
A_I	☐	☐	☐
Commentaires			

Serveur: localhost ▶ Base de données: biblio ▶ Table: Ouvrages

Afficher Structure SQL Rechercher Insérer Exporter Importer Opérations Vider

✓ La table 'biblio'. 'Ouvrages' a été créée.

```
CREATE TABLE `biblio`.`Ouvrages` (
  `Titre` VARCHAR( 100 ) NOT NULL ,
  `Emplacement` CHAR( 2 ) NOT NULL ,
  `ID-Auteur` INT NOT NULL ,
  UNIQUE (
    `Titre`
  )
) ENGINE = INNODB
```

Champ	Type	Interclassement	Attributs	Null	Défaut	Extra	Action
☐ Titre	varchar(100)	utf8_general_ci		Non	aucune		      
☐ Emplacement	char(2)	utf8_general_ci		Non	aucune		      
☐ ID-Auteur	int(11)			Non	aucune		      

Tout cocher / Tout décocher Pour la sélection :      

Version imprimable Gestion des relations Suggérer des optimisations quant à la structure de la table

Ajouter 1 champ(s) ☒ En fin de table ☐ En début de table ☐ Après Titre Exécuter

Index:

Action	Nom de l'index	Type	Unique	Compressé	Champ	Cardinalité	Interclassement	Null	Commentaire
 	Titre	BTREE	Oui	Non	Titre	0	A		

Notre table est prête. Remarquez que j'ai déclaré le champ « Titre » UNIQUE, ce qui signifie que je ne veux pas de doublons et ce champ est automatiquement indexé.

Maintenant remplissons nos tables. Sélectionnez la table « Auteurs » et cliquez sur l'onglet « Insérer » :

phpMyAdmin

biblio (2)

- auteurs
- ouvrages

Serveur: localhost Base de données: biblio Table: auteurs

Afficher Structure SQL Rechercher Insérer Exporter Importer Opérations

Champ	Type	Fonction	Null	Valeur
ID_Auteur	int(11)			
Nom	varchar(100)			

Exécuter

☒ Ignorer

Ce formulaire vous permet de saisir des enregistrements. En entrant « Flaubert Gustave » dans la zone valeur du champ Nom j'obtiens cette page après avoir cliqué sur « Exécuter » :

```

✓ 1 enregistrement(s) inséré(s).
Identifiant de l'enregistrement inséré : 1

INSERT INTO `biblio`.`auteurs` (
  `ID_Auteur` ,
  `Nom`
)
VALUES (
  NULL , 'Flaubert Gustave'
);

```

Vous pouvez voir la syntaxe pour l'insertion d'un enregistrement. Je rentre aussi le grand Hugo de la même manière. En cliquant sur l'onglet « Afficher » on peut lire le contenu de la table :

Serveur: localhost Base de données: biblio Table: auteurs

Afficher Structure SQL Rechercher Insérer Exporter Importer

✓ Affichage des enregistrements 0 - 1 (2 total, Traitement en 0.0006 sec.)

```

SELECT *
FROM `auteurs`
LIMIT 0 , 30

```

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Trier sur l'index: aucune

+ Options

	ID_Auteur	Nom
☐  	1	Flaubert Gustave
☐  	2	Hugo Victor

↑ Tout cocher / Tout décocher Pour la sélection :   

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Vous pouvez lire ici la syntaxe d'une requête de sélection, l'astérisque signifie qu'on désire tous les enregistrements. Remarquez que notre champ d'index s'est correctement incrémenté. De la même manière nous remplissons la table des ouvrages :

Serveur: localhost ▶ Base de données: biblio ▶ Table: ouvrages

Afficher Structure SQL Rechercher Insérer Exporter Importer

✓ Affichage des enregistrements 0 - 4 (5 total, Traitement en 0.0011 sec.)

```
SELECT *
FROM `ouvrages`
LIMIT 0 , 30
```

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Trier sur l'index: aucune

+ Options

			Titre	Emplacement	ID-Auteur
☐			Les misérables	A1	1
☐			L'homme qui rit	A0	1
☐			Madame Bovary	C1	2
☐			Mémoires d'un fou	C2	2
☐			Notre-Dame de Paris	A2	1

↑ Tout cocher / Tout décocher Pour la sélection :

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

3.3 Requêtes SQL

Nous avons vu ci-dessus un certain nombre de requêtes SQL générées automatiquement par **phpMyAdmin**. Mais il va arriver un moment où vous serez obligé d'écrire vous-même des requêtes. Faisons un peu le point des possibilités les plus courantes.

3.3.1 SELECT

La commande la plus souvent utilisée est certainement **SELECT**. Elle permet d'effectuer une sélection au sein des enregistrements. Prenons un exemple simple avec la table des ouvrages. Cherchons les enregistrements dont le champ **Titre** se situe alphabétiquement avant la lettre « M ». La requête correspondant possède une syntaxe simple :

```
SELECT * FROM ouvrages
WHERE ('Titre' < 'M')
```

Cela se lit presque comme du langage naturel : sélectionner (**SELECT**) tous les enregistrements (*) pour lesquels (**WHERE**) le champ **Titre** se situe avant la lettre M (Titre < 'M'). Même si **phpMyAdmin** ne va pas

écrire cette requête à notre place il va quand même nous aider un peu. Une façon de composer une requête consiste à cliquer sur l'onglet « SQL » :

Vous obtenez une zone de saisie vierge pour entrer votre requête. Une différence intervient si vous sélectionnez la table **Ouvrages** avant de cliquer sur l'onglet « SQL » :

Vous obtenez un début de requête puisque **phpMyAdmin** sait dans quelle table vous allez intervenir. D'autre part il vous propose dans la partie droite les champs de cette table. Notre requête devient déjà plus facile à saisir. Mais vous avez une autre possibilité en sélectionnant la base et en cliquant sur l'onglet « Requête » :

Cette fois les possibilités sont plus nombreuses. A vous de choisir ! Quelle que soit votre façon d'entrer la requête vous obtenez ce résultat :

✓ Affichage des enregistrements 0 - 1 (2 total, Traitement en 0.0008 sec.)

```

SELECT *
FROM `ouvrages`
WHERE `Titre` < 'M'
LIMIT 0 , 30

```

☐ Profilage [Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Trier sur l'index: aucune

+ Options

		Titre	Emplacement	ID-Auteur	
☐			Les misérables	A1	1
☐			L'homme qui rit	A0	1

↑ Tout cocher / Tout décocher Pour la sélection :

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Opérations sur les résultats de la requête

Version imprimable
 Version imprimable (avec textes complets)
 Exporter
 CREATE VIEW

Vous constatez que la requête est efficace. Peaufinons un peu en classant par emplacement :

Serveur: localhost
 Base de données: biblio
 Table: ouvrages

✓ Affichage des enregistrements 0 - 1 (2 total, Traitement en 0.0249 sec.)

```

SELECT *
FROM `ouvrages`
WHERE `Titre` < 'M'
ORDER BY `Emplacement`
LIMIT 0 , 30

```

☐ Profilage [Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Trier sur l'index: aucune

+ Options

		Titre	Emplacement	ID-Auteur	
☐			L'homme qui rit	A0	1
☐			Les misérables	A1	1

↑ Tout cocher / Tout décocher Pour la sélection :

Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0

en mode horizontal et répéter les en-têtes à chaque groupe de 100

Opérations sur les résultats de la requête

Version imprimable
 Version imprimable (avec textes complets)
 Exporter
 CREATE VIEW

Vous constatez que j'ai ajouté une clause **ORDER BY** pour effectuer le classement sur le champ **Emplacement**.

Un certain nombre d'opérateurs sont autorisés dans la clause **WHERE** :

Opérateur	Description
=	Egal
<>	Différent
>	Plus grand
<	Plus petit
>=	Plus grand ou égal
<=	Plus petit ou égal
BETWEEN	Entre (inclusif)
LIKE	Comparaison partielle
IN	Recherche dans une plage de valeurs

Voyons maintenant comment extraire des enregistrements concernés par deux tables. Imaginons que je veux obtenir tous les ouvrages de Victor Hugo. Voici cette requête :

```
SELECT `auteurs`.`Nom`, `ouvrages`.`Titre`
FROM auteurs, ouvrages
WHERE ((`auteurs`.`Nom` = 'Hugo Victor') AND (`auteurs`.`ID_Auteur` = `ouvrages`.`ID_Auteur`))
```

La sélection porte que sur deux champs, dans deux tables, la particularité vient de la clause **WHERE** dans laquelle j'ai réalisé une « jointure ». Pour mettre en relation les deux tables j'ai demandé l'égalité des deux champs de liaison. Voyons le résultat de cette requête :

The screenshot shows a database management interface with the following elements:

- Navigation Bar:** Serveur: localhost > Base de données: biblio > Table: auteurs
- Toolbar:** Afficher, Structure, SQL, Rechercher, Insérer, Exporter, Importer, Opérations, Vider.
- Buttons:** Supprimer (with a red X icon).
- Status Bar:** Affichage des enregistrements 0 - 1 (2 total, Traitement en 0.0008 sec.)
- SQL Editor:**

```
SELECT `auteurs`.`Nom` , `ouvrages`.`Titre`
FROM auteurs, ouvrages
WHERE (
  {
    `auteurs`.`Nom` = 'Hugo Victor'
  }
)
AND (
  `auteurs`.`ID_Auteur` = `ouvrages`.`ID_Auteur`
)
)`
```
- Options:**
 - Profilage [Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]
 - Afficher : 30 enregistrement(s) à partir de l'enregistrement n° 0
 - en mode horizontal et répéter les en-têtes à chaque groupe de 100
- Results Table:**

Nom	Titre
Hugo Victor	Madame Bovary
Hugo Victor	Mémoires d'un fou

Vous devez vous douter que je ne vous ai donné qu'un aperçu de cette commande **SELECT**, mais largement suffisante pour vous permettre de faire vos débuts dans ce domaine.

3.3.2 INSERT

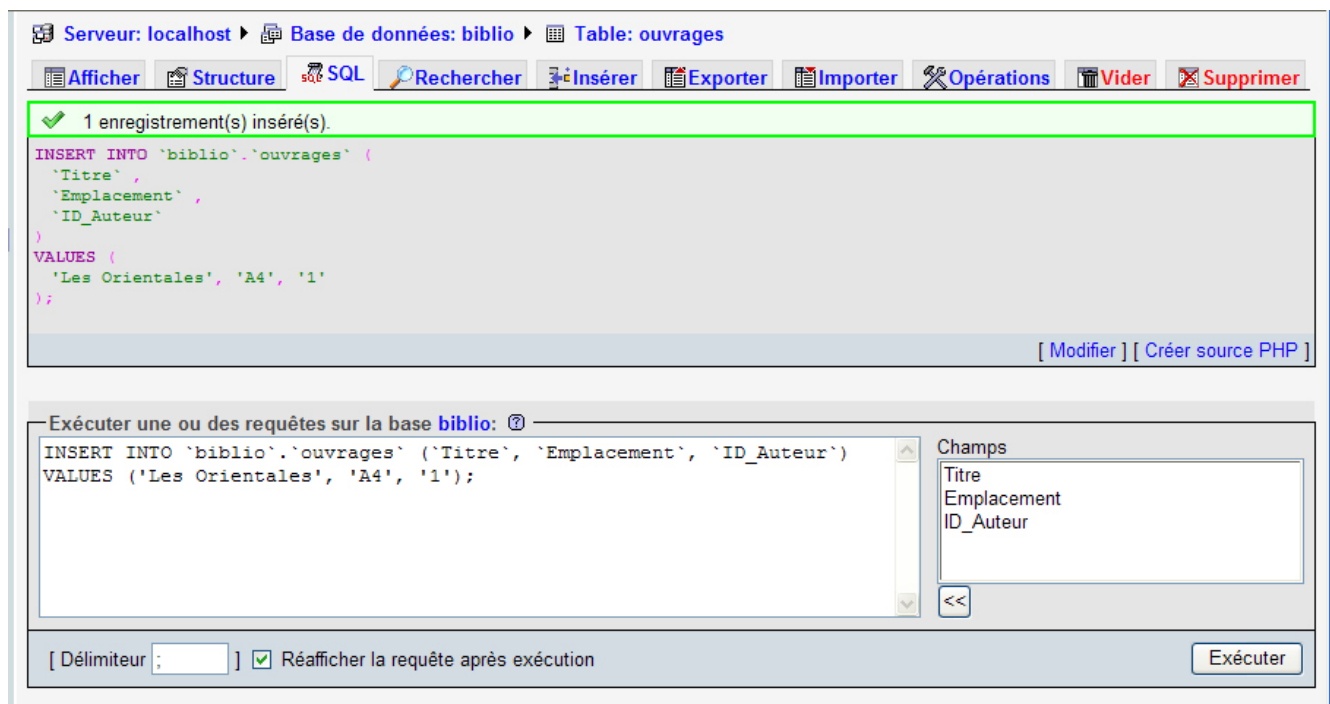
La commande **INSERT** sert à ajouter un ou plusieurs enregistrements. **phpMyAdmin** vous permet d'ajouter un enregistrement facilement, sélectionnez la table **ouvrages** et cliquez sur l'onglet « Insérer ». Faites alors cette saisie :



Champ	Type	Fonction	Null	Valeur
Titre	varchar(100)			Les Orientales
Emplacement	char(2)			A4
ID_Auteur	int(11)			1

Exécuter

Voici le résultat :



✓ 1 enregistrement(s) inséré(s).

```
INSERT INTO `biblio`.`ouvrages` (  
  `Titre` ,  
  `Emplacement` ,  
  `ID_Auteur`  
)  
VALUES (  
  'Les Orientales', 'A4', '1'  
)
```

[Modifier] [Créer source PHP]

Exécuter une ou des requêtes sur la base **biblio**:

```
INSERT INTO `biblio`.`ouvrages` (`Titre`, `Emplacement`, `ID_Auteur`)  
VALUES ('Les Orientales', 'A4', '1');
```

Champs
Titre
Emplacement
ID_Auteur

[Délimiteur :] ☒ Réafficher la requête après exécution Exécuter

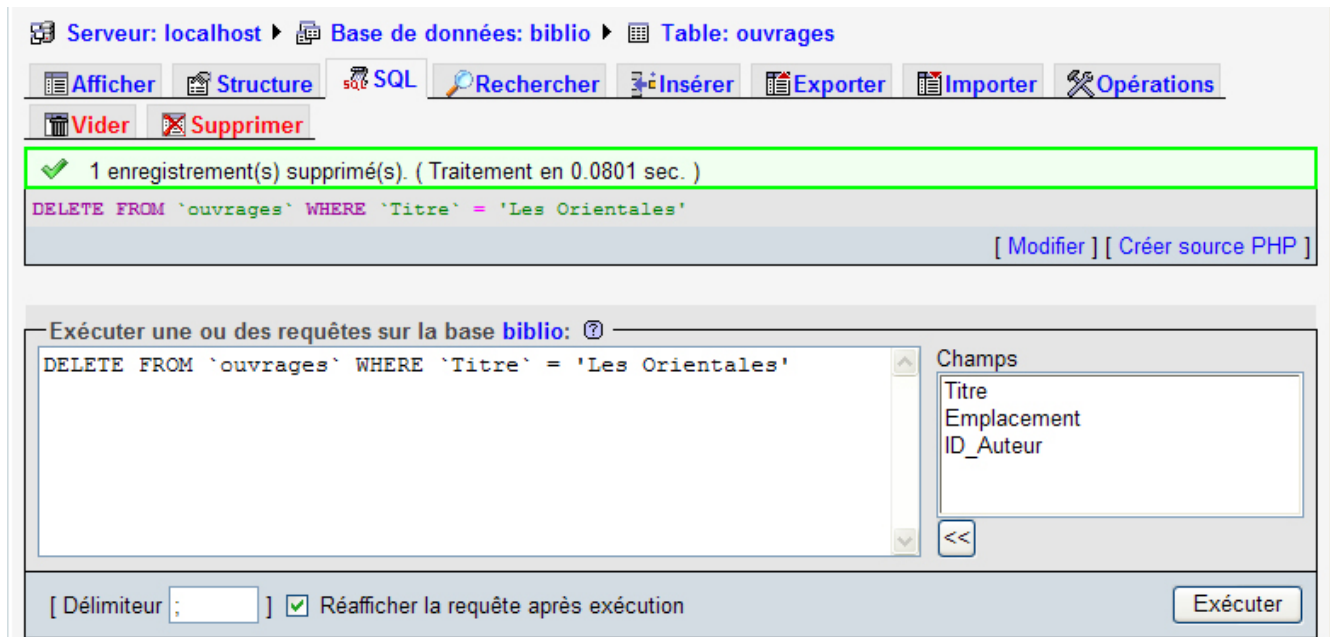
Vous voyez apparaître la syntaxe de la commande **INSERT**. Ici encore c'est très proche du langage naturel : insérer (**INSERT**) dans la table « ouvrages » dans les champs correspondants les valeurs (**VALUES**) saisies.

3.3.3 DELETE

La commande **DELETE** permet de supprimer un ou plusieurs enregistrements. La syntaxe est exactement la même que pour **SELECT**. Je vais supprimer par exemple l'enregistrement ajouté ci-dessus. Voici la requête :

```
DELETE FROM `ouvrages`  
WHERE `Titre` = 'Les Orientales'
```

Résultat :

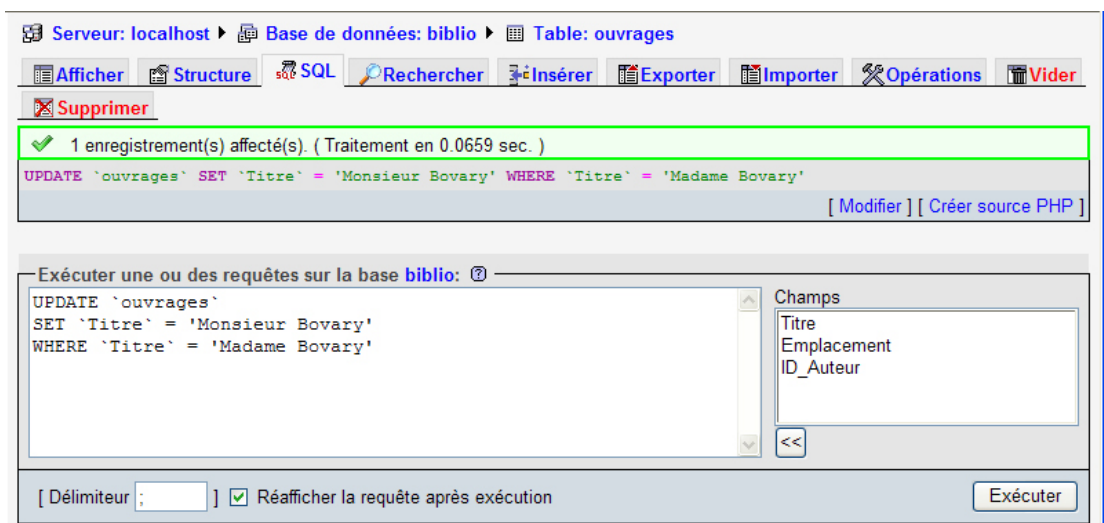


3.3.4 UPDATE

La commande **UPDATE** sert à modifier un ou plusieurs enregistrements. Je vais modifier l'orthographe d'un titre dans la table « ouvrages ». Voici la requête :

```
UPDATE `ouvrages`  
SET `Titre` = 'Monsieur Bovary'  
WHERE `Titre` = 'Madame Bovary'
```

Résultat :



Vous possédez maintenant de bonnes bases avec MySQL.

3.4 Pour aller plus loin

La documentation sur **MySQL** est foisonnante sur le Net. Si vous tapez ce mot dans Google vous obtenez 280 000 000 de réponses et 6 170 000 en se contenant des pages en français.

Vous trouverez la documentation officielle en français ici :

<http://dev.mysql.com/doc/mysql/fr/index.html>

Et ici des tutoriaux en français :

<http://mysql.developpez.com/cours/>

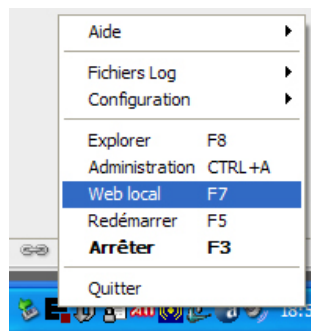
4. PHP

PHP est le langage de script côté serveur le plus célèbre d'Internet. Son principal objectif est de créer des pages **HTML** pour être affichées dans les navigateurs, mais il peut réaliser bien plus que cela (images, animations Flash, fichiers PDF...). Dans le cadre de la réalisation d'un serveur Web pour des applications **LSL** nous allons totalement ignorer ces aspects là pour nous focaliser sur la communication et la gestion d'une base de données. Ce langage en est actuellement à la version 5 mais la plupart des hébergeurs en sont encore à la version 4. **PHP** possède une bibliothèque de fonctions impressionnante (près de 3000) !

Je pars de l'a priori que vous connaissez déjà le **LSL** et la syntaxe de base qui est la même pour le **PHP**. Je vais donc me contenter de préciser les particularités de ce langage. Pour suivre ce chapitre vous aurez besoin d'avoir un serveur **Apache**, un serveur **MySql** et un interpréteur **PHP** sur votre PC, Dans le chapitre sur les outils je vous ai conseillé l'installation d'**EasyPhp**, je vais partir du principe que vous l'avez sur votre ordinateur.

4.1 EasyPhp

Si vous avez laissé **EasyPhp** s'installer dans son répertoire par défaut il doit se trouver dans le répertoire « *C:\Program Files\EasyPHP 3.0* », avec peut-être une différence dans le numéro de version. Vous pouvez trouver un sous-répertoire nommé « *www* » (donc en « *C:\Program Files\EasyPHP 3.0\www\tuto* ». C'est la localisation de votre serveur web local. Je vous conseille la création d'un sous-répertoire nommé « *tuto* » pour placer tous les fichiers que nous allons créer dans ce chapitre. Pour accéder à ce serveur à partir de votre navigateur il faut lancer **EasyPhp** et sélectionner « web local » dans le menu qui apparaît lorsque vous faites un clic droit sur l'icône d' **EasyPhp** :



Je vous ai également conseillé des éditeurs **PHP**. Peu importe celui que vous utilisez, la seule contrainte sera de positionner les fichiers dans le bon répertoire sur votre serveur. Mais vous aurez besoin de la documentation du langage, vous la trouverez sur le site officiel :

<http://www.php.net/>

Il y a une version française à cette adresse :

<http://www.php.net/manual/fr/>

4.2 Les bases

Commençons tranquillement avec ce script :

```
<?php
    echo 'mon premier script';
?>
```

Sauvegardez votre fichier dans le répertoire « tuto » de votre site local. Pour ceux qui connaissent le langage **HTML** (**H**yper**T**ext **M**arkup **L**angage, qui est le langage des pages lues par les navigateurs) la présence d'une balise ne sera pas une surprise. En effet au départ les scripts **PHP** sont intégrés à du code **HTML**. Pour que le serveur sache qu'il s'agit d'un script **PHP** il faut lui signaler au moyen d'une balise. C'est le but de la balise `< ? php>`. Cette balise doit apparaître même si vous ne mettez que du code **PHP** dans un fichier, ce qui sera notre cas tout au long de ce document, **echo** est une commande qui permet d'afficher du texte dans le navigateur, nous allons l'utiliser aussi fréquemment pour afficher des résultats pour nos scripts de test. Lancez le script à partir de votre site local, vous constatez que le texte est affiché dans le navigateur. Transformez le script ainsi :

```
<?php
    echo "mon premier script";
?>
```

Vous constatez que le résultat est le même. Alors les apostrophes et guillemets sont-ils équivalents ? Pas tout à fait. Nous verrons que cela a une importance si on place une variable dans la chaîne, sinon il est juste plus pratique d'utiliser des guillemets si votre chaîne comporte des apostrophes, sinon il faut échapper ceux-ci. Essayez ce script pour comprendre cette différence :

```
<?php
    echo "J'ai raison";
    echo ' d\'avoir raison';
?>
```

4.3 Les variables

Maintenant que nous savons afficher quelque chose voyons un peu ce langage et pour commencer ses variables. Celles-ci commencent avec le signe « \$ » suivi d'un nom sensible à la casse. Mais contrairement à **LSL**, **PHP** est un langage typé dynamiquement, ce qui signifie qu'on ne déclare pas le type des variables, c'est **PHP** qui détermine ce type selon le contexte, et ce type peut changer en cours de script. Il est toujours possible de transtyper une variable pour la forcer d'un certain type mais c'est rarement utile. Regardez ce script (désormais je ne figure plus les balises pour simplifier):

```
$nombre = 2;
$chaine = "toto";
$flottant = 2.3;
echo "Un nombre : $nombre <br>";
echo "Une chaîne : $chaine <br>";
echo "Un flottant : $flottant"
```

A l'exécution on obtient :

```
Un nombre : 2
Une chaîne : toto
Un flottant : 2.3
```

J'ai déclaré trois variables sans déterminer leur type, c'est **PHP** qui le détermine selon leur valeur d'affectation. Voici un tableau des principaux types dans **PHP** :

Type	Description
boolean	TRUE ou FALSE
integer	Entier
double	Virgule flottante
string	Chaîne de caractères
array	Tableau de valeurs

Par rapport à **LSL** on voit apparaître un type **boolean** spécifique et le type **list** est remplacé par le type **array**.

Il y a une différence avec **LSL** au niveau de la portée des variables globales. Avec **LSL** une variable déclarée globalement dans votre script est reconnue dans l'ensemble du script que ce soit les fonctions ou les événements. Pour **PHP** toutes les variables sont locales par défaut. Autrement dit une variable déclarée en dehors des fonctions n'est pas reconnue à l'intérieur de celles-ci. Voici un exemple :

```
$toto = "moi";
function qui() {
    echo $toto;
}
qui();
```

Je déclare une variable globale **\$toto** et je lui affecte la valeur « moi ». Je crée une fonction élémentaire (je décris les fonctions plus loin) qui a pour seul objet d'afficher la valeur de la variable **\$toto** et j'appelle cette fonction. A l'affichage on n'obtient rien. Tout simplement parce que la variable **\$toto** dans la fonction est locale à cette fonction est différente de la variable globale **\$toto**. Voici maintenant le script modifié :

```
$toto = "moi";
function qui() {
    global $toto;
    echo $toto;
}
qui();
```

J'ai déclaré la variable **\$toto** dans la fonction avec le mot clé **global**. J'indique ainsi à PHP que je veux utiliser la variable globale. Maintenant l'affichage devient :

```
moi
```

Un autre type de variable dans PHP : les **superglobales**. Ce sont des variables disponibles de partout. Prenons un exemple avec la superglobale **\$GLOBALS**. Reprenons le script précédent en utilisant cette superglobale :

```
$toto = "moi";
function qui() {
    global $toto;
    echo $GLOBALS["toto"];
}
qui();
```

\$GLOBALS est un tableau associatif (nom de variable => valeur) qui contient toutes les variables globales. Nous verrons d'autres variables superglobales plus loin.

4.4 Les tableaux

L'équivalent des **list** de **LSL** sont les **array**. Basiquement ce sont des tableaux pouvant recevoir n'importe quel type de donnée, le tout indexé.

4.4.1 Tableaux indicés

Les tableaux indicés ne vous poseront pas beaucoup de problèmes parce qu'ils sont vraiment très proches des **list**. Chaque élément du tableau est repéré par un nombre en partant de 0. Il y a plusieurs manières de remplir un tableau indicé :

```
$tableau = array ("chat","chien","poulet");
$tableau[3] = "perroquet";
$tableau[] = "coq";
echo "$tableau[0]<br>";
echo "$tableau[1]<br>";
echo "$tableau[2]<br>";
echo "$tableau[3]<br>";
echo "$tableau[4]<br>";
```

Résultat :

```
chat
chien
poulet
perroquet
coq
```

4.4.2 Tableaux associatifs

Si les tableaux indicés ne sont pas trop dépaynants pour vous il en ira peut-être différemment avec les tableaux associatifs. Dans ce type de tableau au lieu d'un indice chiffré on a une **clé** qui peut être un nombre ou une chaîne de caractères. Regardez cet exemple :

```
$tableau["peintre"] = "alfred";
$tableau["carreleur"] = "albert";
$tableau["couvreur"] = "marcel";
echo $tableau["peintre"] , "<br>";
echo $tableau["carreleur"] , "<br>";
echo $tableau["couvreur"] , "<br>";
```


Sortie :

```
alfred  
albert  
marcel
```

Voici une autre syntaxe pour un résultat identique :

```
$tableau = array (  
"peintre" => "alfred",  
"carreleur" => "albert",  
"couvreur" => "marcel");  
echo $tableau["peintre"] , "<br>";  
echo $tableau["carreleur"] , "<br>";  
echo $tableau["couvreur"] , "<br>";
```

4.4.3 Fonctions pour les tableaux

Les tableaux sont très fréquemment employés et de nombreuses fonctions sont prévues pour les manipuler. Voici un tableau des principales :

Fonction	Description
reset	Positionne le pointeur au début du tableau et retourne sa valeur
prev	Reculé le pointeur du tableau et retourne sa valeur
next	Avance le pointeur du tableau et retourne sa valeur
end	Positionne le pointeur en fin du tableau et retourne sa valeur
each	Retourne la paire clé/valeur et avance le pointeur
current	Retourne la paire clé/valeur sans avancer le pointeur
count	Retourne le nombre d'éléments du tableau
key	Retourne la clé de la position courante du pointeur
foreach	Comme each mais en plus passe en boucle tous les éléments
in_array	Indique si une valeur est dans un tableau
sort	Tri des valeurs du tableau
rsort	Tri inverse des valeurs du tableau
asort	Tri des valeurs du tableau en gardant clé/valeur
arsort	Tri inverse des valeurs du tableau en gardant clé/valeur
natsort	Tri « naturel » du tableau (ex : n1, n13, n2)
array_slice	Extrait une partie d'un tableau

La notion de pointeur de tableau n'existe pas dans **LSL**. Avec **PHP** un pointeur permet de parcourir un tableau et les fonctions **reset**, **prev**, **next**, **end**, permettent de gérer efficacement ce pointeur. Regardez ce code :

```
$tableau = array ("peintre" => "alfred", "carreleur" => "albert", "couvreur" => "marcel");  
echo current($tableau), "<br>";  
echo next($tableau), "<br>";  
echo reset($tableau), "<br>";  
echo end($tableau), "<br>";
```

```
echo prev($tableau), "<br>";
```

Sortie :

```
alfred  
albert  
alfred  
marcel  
albert
```

La structure de contrôle **foreach** permet de parcourir les éléments d'un tableau. Sous **LSL** vous êtes obligé d'utiliser une variable pour parcourir les index, avec **PHP** la syntaxe est plus simple :

```
$tableau = array ("peintre" => "alfred", "carreleur" => "albert", "couvreur" => "marcel");  
foreach ($tableau as $valeur)  
    echo $valeur . "<br>";
```

Sortie :

```
alfred  
albert  
marcel
```

Il est possible aussi de récupérer les clés et les valeurs:

```
$tableau = array ("peintre" => "alfred", "carreleur" => "albert", "couvreur" => "marcel");  
foreach ($tableau as $clef => $valeur)  
    echo $clef . " => " . $valeur . "<br>";
```

Sortie :

```
peintre => alfred  
carreleur => albert  
couvreur => marcel
```

Une chose est à savoir par rapport à l'instruction **foreach** : elle utilise une copie du tableau passé en paramètre. Ce qui signifie que vous ne pouvez pas modifier les valeurs du tableau d'origine avec cette instruction. Regardez le script suivant :

```
$tableau = array("alfred", "albert", "marcel");  
foreach ($tableau as $valeur)  
    $valeur = strtoupper($valeur);  
foreach ($tableau as $valeur)  
    echo $valeur . "<br>";
```

Dans une première boucle le force tous les caractères dans valeur en majuscules avec la fonction **strtoupper**. Mais la deuxième boucle qui me sert à afficher le tableau maintient un affichage en minuscules. On a donc bien la démonstration qu'on travaille sur une copie du tableau. Voici le script modifié :

```
$tableau = array("alfred", "albert", "marcel");  
foreach ($tableau as &$valeur)  
    $valeur = strtoupper($valeur);  
print_r($tableau);
```

Vous remarquez une petite différence avec l'ajout du caractère « & » devant la variable **\$valeur**. On indique à **PHP** qu'on veut les valeurs par référence, donc pointer sur les véritables valeurs dans le tableau. Au passage la fonction **print_r** permet d'afficher le type et le contenu d'une variable. Voici la sortie :

```
Array ( [0] => ALFRED [1] => ALBERT [2] => MARCEL )
```

Cette fois les valeurs on été transformées comme on le souhaitait. Mais cette syntaxe ne fonctionne qu'à partir de la version 5 de **PHP**.

Voici un script utilisant quelques fonctions consacrées aux tableaux :

```
$tableau = array("alfred", "albert", "marcel");  
echo "Nombre de valeurs : " . count($tableau) . "<br>";  
echo "Tableau classé croissant : ";  
sort($tableau);  
print_r($tableau);  
rsort($tableau);  
echo "<br>", "Tableau classé décroissant : ";  
print_r($tableau);  
echo "<br>", "Extraction d'un sous-tableau : ";  
print_r(array_slice($tableau, 1, 2));
```

Et voici le résultat :

```
Nombre de valeurs : 3  
Tableau classé croissant : Array ( [0] => albert [1] => alfred [2] => marcel )  
Tableau classé décroissant : Array ( [0] => marcel [1] => alfred [2] => albert )  
Extraction d'un sous-tableau : Array ( [0] => alfred [1] => albert )
```

4.5 Chaînes de caractères

Le traitement des chaînes de caractères est pratiquement identique à celui qui est opéré avec **LSL**. Il faut toutefois signaler une différence au niveau de la concaténation, ce n'est pas l'opérateur « + » qui est utilisé mais « . ». regardez ce script :

```
$chaine1 = "cara";  
$chaine2 = "mel";  
$chaine3 = $chaine1.$chaine2;  
echo "$chaine3";
```

qui donne à l'exécution :

```
caramel
```

Si vous essayez avec l'opérateur « + » vous obtenez 0 parce que **PHP** considère alors les opérandes comme des valeurs numériques nulles.

PHP propose de nombreuses fonctions pour traiter le type **string**. Voici un tableau des plus importantes :

Fonction	Description
explode	Découpe une chaîne en morceaux
implode	Rassemble des éléments de tableau dans une chaîne
ltrim	Supprime des caractères ou espaces en début de chaîne
rtrim	Supprime des caractères ou espaces en fin de chaîne
strtr	Remplace des caractères
str_shuffle	Mélange les caractères
str_split	Convertit une chaîne en tableau
strlen	Calcule le nombre de caractères
strpos	Trouve l'emplacement d'une sous-chaîne
strrev	Inverse la chaîne
substr	Retourne un segment de chaîne

Voici un script qui utilise quelques unes de ces fonctions :

```
$alphabet = "abcdefghijklmnopqrstuvwxyz";  
echo "Longueur de la chaîne : ", strlen($alphabet), "<br>";  
echo "Chaîne mélangée : ", str_shuffle($alphabet), "<br>";  
echo "La lettre h dans l'alphabet est en position : ", strpos($alphabet, "h"), "<br>";  
echo "Chaîne inversée : ", strrev($alphabet), "<br>";  
echo "Extraction chaîne : ", substr($alphabet, 7, 10), "<br>";  
echo "Remplacement de chaîne : ", strtr($alphabet, array("d" => "t", "lm" => "cv")), "<br>";  
$tabSegments = explode("h", $alphabet);  
echo "Segmentation de chaîne : ";  
print_r($tabSegments);  
echo "<br>", "Rassemblement de chaîne : ";  
print_r(implode("h", $tabSegments));
```

Affichage :

```
Longueur de la chaîne : 26  
Chaîne mélangée : cqtxvgjewnidmozakhrupsfbyl  
La lettre h dans l'alphabet est en position : 7  
Chaîne inversée : zyxwvutsrqponmlkjihgfedcba  
Extraction chaîne : hijklmnopq  
Remplacement de chaîne : abctefghijkcvnopqrstuvwxyz  
Segmentation de chaîne : Array ( [0] => abcdefg [1] => iklmnopqrstuvwxyz )  
Rassemblement de chaîne : abcdefghijklmnopqrstuvwxyz
```

4.6 Les fonctions

PHP permet évidemment la création de fonctions utilisateur avec des possibilités bien plus étendues que **LSL**. Mais les fonctions sont comme les variables : non typées. Dans **LSL** vous définissez une fonction en écrivant un type et un nom. Dans **PHP** vous utilisez le mot clé **function** suivi d'un nom. Comme dans **LSL** le nombre d'arguments de la fonction est libre mais ces arguments ne sont pas typés. Voici un exemple simple de fonction et de son utilisation :

```
function affiche($nombre)
{
    echo number_format($nombre, 2, ',', ' ');
}
affiche(12.3525);
```

Affichage :

```
12,35
```

La fonction a pour objectif d'afficher un nombre au format virgule et deux décimales. Au passage vous découvrez la fonction **number_format** bien pratique ! La fonction a un seul argument et ne renvoie aucune valeur. Regardez le script transformé avec une valeur de retour :

```
function affiche($nombre)
{
    return number_format($nombre, 2, ',', ' ');
}
echo affiche(12.3525);
```

Encore une fois le type est donné par le contexte. Regardez maintenant une nouvelle transformation :

```
function affiche($nombre = 65.324)
{
    return number_format($nombre, 2, ',', ' ');
}
echo affiche(12.3525), "<br>";
echo affiche();
```

Affichage :

```
12,35
65,32
```

L'argument a une valeur par défaut ce qui fait que lorsqu'on appelle la fonction sans transmettre de valeur c'est cette valeur par défaut qui est prise en compte. Passons à un autre aspect :

```
function double($a)
```

```
{
    $a *= 2;
    return $a;
}
$a = 6;
echo double($a), " est le double de ", $a, "<br>";
```

Affichage :

```
12 est le double de 6
```

Faites cette petite modification dans le script :

```
function double(&$a)
{
    $a *= 2;
    return $a;
}
$a = 6;
echo double($a), " est le double de ", $a, "<br>";
```

Maintenant vous obtenez :

```
12 est le double de 12
```

Que c'est-il passé ? Le fait d'ajouter le caractère « & » devant la variable la transforme en référence. Dans **LSL** vous ne connaissez que le passage des paramètres par valeur, autrement dit la variable est strictement locale à la fonction, c'est juste la valeur qui est transmise. Dans une transmission par référence la variable pointe dans la même zone mémoire et si elle est modifiée dans la fonction elle l'est aussi dans le contexte global.

Pour le passage des arguments **PHP** pousse la souplesse jusqu'à permettre un nombre indéfini d'arguments. Observez le script suivant :

```
function somme()
{
    $t = func_get_args();
    return array_sum($t);
}
echo somme(2, 5, 3, 56);
```

Affichage :

```
66
```

La fonction **somme** est déclarée sans argument. Pourtant lors de son appel je passe 4 valeurs numériques et j'obtiens bien leur somme. La fonction **func_get_args** renvoie un tableau des valeurs passées. J'applique la fonction **array_sum** qui ajoute toutes les valeurs contenues dans un tableau et je retourne le résultat.

4.7 Les structures de contrôle

Les structures de contrôle dans **PHP** sont les mêmes que dans **LSL** mis à part l'instruction **switch** qui manque dans **LSL**. Voici un exemple de son utilisation :

```
$n = mt_rand(1, 3);
echo "Le nombre est ";
switch ($n)
{
    case 1:
        echo "1";
        break;
    case 2:
        echo "2";
        break;
    default:
        echo "3";
}
```

La fonction **mt_rand** fournit un nombre entier aléatoire, dans notre cas entre 1 et 3. L'instruction **switch** teste la valeur de la variable et traite plusieurs cas (**case**). Dans chaque cas testé positivement le code qui suit est exécuté. L'instruction **break** permet de sortir du **switch**, sinon les instructions suivantes sont exécutées. Cette instruction est une alternative à la suite de **else if**.

4.8 Les objets

PHP autorise la programmation par objets. La version 4 propose des possibilités limitées alors que la version 5 expose toutes les subtilités de cette approche. Mais d'abord qu'est-ce que la **POO** (Programmation Orientée Objet) ? Plutôt que de programmer classiquement on crée des objets genre « boîtes noires » qui exposent à l'extérieur des **attributs** et des **méthodes**, et ils se débrouillent pour faire leur cuisine interne pour que ça fonctionne. Ce style de programmation est plus naturel et permet une meilleure organisation du code.

Prenons un exemple. Imaginez que vous créez des boîtes de différentes formes (cubes et parallélépipèdes). Qu'est-ce qui différencie deux cubes ? Leur taille, leur couleur par exemple. Ce sont des **attributs** des cubes. On pourrait demander à un cube de nous indiquer son volume, c'est une **méthode**. On va différencier la notion générale de cube, qu'on appelle **classe**, et les cubes eux-mêmes qui sont des **instances** de la classe, des **objets**. Les **instances** se différencient entre elles par des différences dans les valeurs des **attributs**. Par exemple si on définit l'**attribut** « longueur d'un côté » dans la **classe** cube, chaque **instance** va avoir une valeur particulière pour cet **attribut**. Il en est de même si on définit un **attribut** « couleur ». On peut avoir le même raisonnement pour les parallélépipèdes avec des attributs différents mais certains communs, comme la couleur. On se rend compte que les **attributs** communs impliquent la notion de boîte qui est plus générale que celle de cube ou parallélépipèdes. On peut créer une **classe** « boîte » avec des **attributs** et des **méthodes**. Les **classes** « cube » et « parallélépipède » peuvent alors **hériter** de cette **classe** les **attributs** et **méthodes** communs et ajouter leur propre fonctionnement. La **classe** boîte n'a alors pas pour vocation l'**instanciation** d'objets.

Après ces rapides généralités sur la **POO** je vais prendre un exemple simple pour voir le codage correspondant. Puisque nous traitons de SL considérons la **classe** « avatar ». Les attributs sont : la clé, le nom. Je vais distinguer le codage dans les versions 4 et 5 de **PHP** parce qu'elles ont des différences sensibles.

4.8.1 Version PHP4

Considérez ce code :

```

class avatar
{
    var $nom;
    var $clé;
    function direNom()
    {
        echo "Je m'appelle " . $this->nom;
    }
}
$toto = new avatar();
$toto->nom = "Alex Terieur";
$toto->clé = "66864f3c-e095-d9c8-058d-d6575e6ed1b8";
$toto->direNom();

```

Je crée une classe « avatar » avec le mot clé **classe**. Je définis deux **attributs** : « nom » et « clé » avec le mot clé **var**. Je définis aussi une **méthode** de la même manière qu'on définit une fonction. **\$this** est une métavariable qui référence automatiquement l'objet courant, on peut ainsi faire appel aux variables de l'objet courant, ici la variable **nom**. Le mot clé **new** sert à créer une instance de la classe concernée. Il est possible de créer une méthode particulière qui porte le même nom que la classe qui est automatiquement appelée à la création d'un objet, on dit que c'est un **constructeur**. Voici le code modifié :

```

class avatar
{
    var $nom;
    var $clé;
    function avatar($nom, $clé)
    {
        $this->nom = $nom;
        $this->clé = $clé;
    }
    function direNom()
    {
        echo "Je m'appelle " . $this->nom;
    }
}
$toto = new avatar("Alex Terieur", "66864f3c-e095-d9c8-058d-d6575e6ed1b8");
$toto->direNom();

```

4.8.2 Version PHP5

PHP dans sa version 5 améliore la prise en charge de la **POO**. Voici l'équivalent du script précédent :

```

class avatar
{
    private $nom;
    private $clé;
    function __construct($nom, $clé)
    {
        $this->nom = $nom;
    }
}

```



```

    $this->clé = $clé;
}
function direNom()
{
    echo "Je m'appelle " . $this->nom;
}
}
$toto = new avatar("Alex Terieur", "66864f3c-e095-d9c8-058d-d6575e6ed1b8");
$toto->direNom();

```

Vous pouvez constater deux différences. La première concerne la déclaration des attributs, ils se font avec le mot clé **private**. Ce qui signifie que l'attribut ne peut pas être atteint de l'extérieur de l'objet. On aurait pu utiliser le mot clé **public** pour obtenir un fonctionnement équivalent au précédent mais dans une optique **POO** il est plus cohérent de rendre les attributs inaccessibles directement de l'extérieur. Avec **PHP4** on n'a pas le choix, il n'y a que le mot clé **var** et il est équivalent au **public** de **PHP5**. Une autre différence réside dans le nom du constructeur, pour **PHP5** c'est toujours le mot clé **__construct**. Ca évite certains problèmes lors de l'héritage d'une classe.

Il y a bien d'autres différences entre les deux versions mais je m'en tiendrai là pour cette petite présentation.

4.9 FTP

Pour transférer vos fichiers sur votre serveur vous aurez besoin d'un client **FTP**. Le **FTP** (File Transfert Protocol) est le protocole principal pour le transfert et de gestion de fichiers sur Internet. La version « client » est celle que vous devez installer sur votre ordinateur. Parmi toute l'offre de programmes c'est certainement **FileZilla** la plus intéressante. Le programme est puissant, pratique, en version Française, et gratuit. Vous pouvez le trouver ici :

<http://www.filezilla.fr/>

Vous trouverez également sur ce site quelques explications en Français pour bien démarrer. Le programme ressemble un peu à l'explorateur de Windows mais vous avez les répertoires et fichiers du serveur dans l'une des fenêtres et les répertoires et fichiers de votre ordinateur dans une autre fenêtre. Il suffit de faire glisser des fichiers d'une fenêtre à l'autre pour opérer un transfert. Vous pouvez aussi effacer, renommer des fichiers directement sur le serveur. La phase la plus délicate est celle du paramétrage de votre connexion. Il vous faut connaître le nom de l'hôte, votre identifiant et votre mot de passe. Vous pouvez récupérer tout ça chez votre hébergeur. Par exemple le nom d'hôte chez Free est « ftperso.free.fr ».

4.10 Pour aller plus loin

La documentation sur **PHP** sur Internet est impressionnante. Lorsque je tape **PHP** dans Google j'obtiens 9 650 000 000 réponses. Un bon point de départ ici avec des tutoriaux progressifs :

<http://www.phpdebutant.org/>

Vous trouverez également pas mal de tutoriaux ici :

<http://php.developpez.com/cours/>

Et ici une source très complètes de scripts pour toutes applications ainsi que bouts de codes et tutoriaux (même si les liens ne sont pas toujours à jour) :

<http://www.phpscripts-fr.net/>

Il y a de très nombreux ouvrages sur le sujet mais celui-ci est vraiment adapté à une initiation :

« Je débute en PHP4 » par Bill McCarty, éditions FIRST

5. Communication

Un des principaux atouts de l'informatique est sa capacité à faire communiquer des systèmes entre eux. Je vais regrouper dans ce chapitre, de manière un peu non conventionnelle, tout ce qui correspond à ce concept, quel que soit la localisation et la nature des systèmes concernés.

5.1 Liaisons entre scripts de PHP

Avec **LSL** vous avez peut-être l'habitude de faire communiquer des scripts entre eux au moyen de **linkMessages** ou même au travers des canaux du Chat. La mémoire allouée à un script avec **LSL** est limitée à 64Ko en mono. Pour certaines applications gourmandes cette limite est vite atteinte. D'autre part il est pratique de découper une application en plusieurs scripts par souci d'organisation du code. Avec **PHP** on ne rencontre pas de limite de mémoire pour un script et pour des applications simples un seul script est largement suffisant. Mais pour les projets plus importants on se retrouve rapidement avec un certain nombre de scripts, comment arrivent-ils à communiquer entre eux ? Ce sont les fonctions **require** et **include** qui permettent cette liaison. Créez ce premier script et appelez-le **inclusion.inc.php** :

```
echo "Je suis là !";
```

Crée maintenant ce second script :

```
echo "On peut inclure un fichier.", "<br>";  
include('inclusion.inc.php');  
echo "<br>", "C'est très facile...";
```

Lancez ce deuxième script, vous obtenez :

```
On peut inclure un fichier.  
Je suis là !  
C'est très facile...
```

Vous voyez bien que l'instruction du fichier inclus fonctionne correctement dans le script. On peut donc créer des bibliothèques de code, des groupes de constantes, rassemblés dans des fichiers que l'on inclut au besoin. On dispose pour ce faire des deux fonctions : **include** et **require**. La première réévalue à chaque fois les instructions alors que la seconde se contente de conserver les données pour un autre passage éventuel. Il existe aussi les fonctions **include_once** et **require_once**. Comme leur nom l'indique clairement elles ne permettent l'inclusion qu'une seule fois, aux passages suivants rien ne survient.

5.2 Liaisons entre PHP et MySql

Les concepteurs de **PHP** ont tout prévu pour gérer une base de donnée **MySql**. Ici aussi il va me falloir faire une différence entre les versions 4 et 5 de **PHP**, en effet la version 5.1 a introduit de grands bouleversements dans ce domaine comme je vais vous le montrer ci-après. Je vais reprendre la base « biblio » que j'ai décrite dans le chapitre sur **MySql** pour les exemples.

5.2.1 Version PHP4

La première chose à faire est de se connecter à la base de données :

```
// Paramètres
$server = 'localhost';
$user = 'root';
$pass = 'mysql';
$base = 'biblio';
// Connexion
$link = mysql_connect($server, $user, $pass);
if (!$link)
    die('Impossible de se connecter : ' . mysql_error());
// Choix de la base
$db_selected = mysql_select_db($base);
if (!$db_selected)
    die('Impossible de trouver la base : ' . mysql_error());
```

Vous avez évidemment besoin de connaître le serveur, le nom de l'utilisateur et son mot de passe pour vous connecter. Ici j'ai entré les valeurs d'un serveur local avec les paramètres par défaut de **MySQL**. La connexion se fait avec la fonction **mysql_connect** qui renvoie l'identification de connexion ou **FALSE** en cas d'échec. Si on n'arrive pas à se connecter on arrête le script avec la fonction **die** et on affiche l'erreur rencontrée avec la fonction **mysql_error**. Si la connexion a réussi on sélectionne la base avec la fonction **mysql_select_db**. Ici aussi on teste l'échec.

Maintenant qu'on est connecté on peut utiliser une requête sur la base. Par exemple :

```
// Requête
$query = "SELECT Titre FROM ouvrages";
$result = mysql_query($query);
if (!$result)
    die('Impossible d\'exécuter la requête : ' . mysql_error());
// Affichage des résultats
while ($row = mysql_fetch_row($result))
    echo $row[0], '<br>';
// Libération des ressources SQL
mysql_free_result($result);
// Déconnexion
mysql_close($link);
```

Affichage :

```
Les misérables
L'homme qui rit
Madame Bovary
Mémoires d'un fou
Notre-Dame de Paris
```

L'appel d'une requête se fait avec la fonction **mysql_query**, on teste également l'échec éventuel de la requête. Il suffit ensuite d'afficher les résultats. Ici j'utilise la fonction **mysql_fetch_row** qui renvoie chaque enregistrement

résultant sous forme de tableau indicé. Les lignes qui suivent sont de la pure intendance, histoire de travailler proprement. A la fin d'un script **PHP** libère automatiquement les ressources et déconnecte de **MySQL** mais c'est quand même plus propre comme ça.

Il n'y a pas que la fonction **mysql_fetch_row** pour récupérer les résultats d'une requête. Vous pouvez aussi utiliser la fonction **mysql_fetch_assoc**. Dans ce cas c'est un tableau associatif qui est retourné :

```
while ($row = mysql_fetch_assoc($result))  
    echo $row['Titre'], '<br>';
```

Si vous préférez travailler avec des objets alors utilisez la fonction **mysql_fetch_object** :

```
while ($row = mysql_fetch_object($result))  
    echo $row->Titre, '<br>';
```

PHP propose toujours de multiples manières d'arriver à un résultat, à vous de trouver votre style.

5.2.2 Version PHP5

PHP5 introduit la notion de **PDO (PHP Data Object)**. C'est un outil permettant de connecter un grand nombre de bases de données : MySQL, Oracle, PostgreSQL, SQL Server. Avec **PDO** on peut exécuter toutes les requêtes **SQL** classiques et même des requêtes mémorisées dans la base de données. Voyons comment on code cela :

```
// Paramètres  
$user = 'root';  
$pass = 'mysql';  
$dsn = 'mysql:host=localhost;dbname=biblio';  
// Connexion  
try  
{  
    $dbh = new PDO($dsn, $user, $pass);  
}  
catch (PDOException$e)  
{  
    echo 'Impossible de se connecter : ' . $e->getMessage();  
}  
// Requête  
$query = "SELECT Titre FROM ouvrages";  
$result = $dbh->exec($query);
```

Mais étant donné qu'il y a peu de chance que votre hébergeur vous autorise cette extension je n'insisterai pas sur cette possibilité.

5.2.3 Protocole HTTP

Le protocole HTTP (**H**yper **T**ext **T**ransfert **P**rotocol) est le principal protocole d'échange de données sur le Web. C'est celui que vous utilisez lorsque vous surfez avec votre navigateur. C'est un protocole client/serveur synchrone : c'est le client qui initialise l'échange et le serveur lui répond. Ce protocole en est actuellement à la version 1.1. Une requête HTTP possède 3 parties :

1. Introduction

2. En-tête
3. Corps

5.2.3.1 Requête du client

Le principal élément de l'introduction est la méthode employée. Voici les principales :

Méthode	Description
GET	Méthode la plus courante, simple téléchargement d'un document mais possibilité d'ajouter des paramètres
POST	Méthode de base pour envoyer du contenu au serveur
PUT	Demande au serveur de mémoriser le contenu envoyé
DELETE	Suppression d'un fichier sur le serveur

L'introduction comporte aussi le chemin vers la ressource du serveur. Il s'agit en fait d'une **URL** (**Uniform Ressource Locator**), comme son nom l'indique lorsqu'on utilise une **URL** on doit obtenir toujours le même résultat en réponse, sauf mise à jour des informations entre temps. Ces **URL** apparaissent dans la zone d'adresse de votre navigateur. Par exemple :

```
http://www.google.com/search?q=sapin
```

Dans cette **URL** on voit facilement l'adresse de la ressource, ici la page « search » sur le serveur de Goggle, et un paramètre transmis dont le nom est « q » et la valeur « sapin ». c'est tout simplement une requête au moteur de Goggle pour trouver les pages où figure le mot « sapin ». Remarquez la syntaxe : le « ? » qui sépare la page des paramètres et l'opérateur « = » entre le nom du paramètre et sa valeur. La méthode utilisée dans ce cas est **GET**. Regardez maintenant cette **URL** :

```
http://www.google.com/search?hl=fr&q=r%C3%A9veil
```

C'est encore une requête adressée à la même ressource mais on voit apparaître des caractères étranges au niveau des paramètres. Vous constatez déjà qu'il y a deux paramètres « hl » et « q ». La séparation entre les deux est réalisée par le caractère « & ». La valeur du deuxième paramètre comporte des caractères spéciaux et mérite une explication. Le problème est qu'une URL ne supporte pas certains caractères réservés comme ?, =, &, #, %, :, et les caractères accentués. Lorsque je tape le mot « réveil » le caractère « é » est transformé (on dit encodé) pour être transmis, vous constatez qu'il s'est transformé en « %C3%A9 ». Il faudra bien entendu le décoder à l'arrivée.

Pour une requête **GET** le corps est vide. Par contre pour les méthodes **POST** et **PUT** le corps comporte les données à traiter.

5.2.3.2 Reponse du serveur

La réponse du serveur comporte des informations intéressantes en plus des informations demandées :

Code	Description
200 OK	Tout s'est bien passé
201 Created	Le document a été créé (par exemple avec une méthode PUT)
400 Bad Request	Requête non valide
403 Forbidden	Ressource interdite pour le client
404 Not Found	Ressource non trouvée
405 Method Not Allowed	Méthode non autorisée pour le client

500 Internal Server Error	Erreur interne au serveur
501 Not Implemented	Certaines fonctionnalités ne sont pas supportées
503 Service Unavailable	Serveur surchargé

5.2.3.3 Traitement par PHP

Les paramètres reçus par la méthode **GET** sont placés dans le tableau superglobal **\$_GET**. Observons ce fonctionnement avec un simple formulaire. Créez la page **HTML** suivante (du simple texte et sauvegarde avec le suffixe « .html »):

```
<form method="get" action="traitement.php">
  <label>Nom :
  <input type="text" name="nom">
</label>
<p>
  <label>Envoyez
  <input type="submit" name="button" value="Envoyer">
  </label>
</p>
</form>
```

Mettez là sur votre serveur local. Lorsque vous la lancez vous obtenez ce formulaire :



Une zone pour saisir un texte et un bouton pour l'envoyer. Vous remarquez dans la balise `<form>` que j'ai utilisé la méthode **GET**. Vous voyez également que je déclare le fichier de traitement « traitement.php ». Créons maintenant ce fichier :

```
echo 'Le nom est : ' . $_GET['nom'];
```

Le nom est récupéré dans la superglobale **\$_GET**. Sauvegardez le sur votre serveur dans le même répertoire que le formulaire. Entrez un nom dans le formulaire et cliquez sur le bouton, vous devriez obtenir un résultat de ce genre :

Le nom est : toto

Si vous regardez l'URL vous devriez avoir un texte dans ce genre :

http://127.0.0.1/traitement.php?nom=toto&button=Envoyer

Vous retrouvez la syntaxe vue précédemment avec le nom du fichier de traitement, le séparateur « ? », et deux paramètres dont le premier est le nom saisi. C'est votre navigateur qui a créé cette requête. Utilisons maintenant la méthode **POST** avec le même formulaire :

```
<form method="post" action="traitement.php">
<label>Nom :
<input type="text" name="nom">
</label>
<p>
<label>Envoyez
<input type="submit" name="button" value="Envoyer">
</label>
</p>
</form>
```

Il faut évidemment modifier le code PHP pour lire dans une autre superglobale :

```
echo 'Le nom est : ' . $_POST['nom'];
```

Nous obtenons un résultat identique par contre en lisant l'URL :

```
http://127.0.0.1/traitement.php
```

On ne voit plus de paramètres puisque avec la méthode **POST** les informations sont contenues dans le corps de la requête, donc invisibles. Cette méthode s'impose pour la transmission de données confidentielles ou volumineuses. Il y a une autre façon de récupérer une donnée transmise par **GET** ou **POST** :

```
echo 'Le nom est : ' . $_REQUEST['nom'];
```

C'est une superglobale qui englobe **\$_GET** et **\$_POST**. Elle fonctionne donc quelle que soit la méthode utilisée. Au lieu d'utiliser un formulaire on pourrait passer par du code **PHP** pour créer la requête :

```
$nom = urlencode('toto');
$lien = "traitement.php?nom=$nom";
echo "<a href='$lien'>Cliquer ici</a>"
```

Si vous lancez ce script sur votre serveur vous obtenez un lien dans votre navigateur, si le script est dans le même répertoire que le fichier « traitement.php » vous allez obtenir le même comportement que précédemment, la différence est que l'URL a été construite par **PHP**. La fonction **urlencode** permet l'encodage des caractères spéciaux qui ne sont pas digérés par **HTTP**. Dans notre cas ça ne sert à rien mais il vaut mieux l'utiliser systématiquement pour éviter les problèmes.

5.2.3.4 Traitement par LSL

La fonction pour envoyer une requête **HTTP** est **llHttpRequest**. Cette fonction comporte 3 paramètres :

Numéro	Paramètre	Type	Description
1	url	string	URL de destination
2	parameters	list	Liste des paramètres de configuration de la forme

			[HTTP..., valeur...]
3	body	string	Corps de la requête

Par défaut c'est la méthode **GET** qui est utilisée, dans ce cas il est inutile de préciser le paramètre **parameters**. Par contre si vous voulez utiliser la méthode **POST** il faut prévoir la liste [HTTP_METHOD, "POST"]. De la même manière avec la méthode **GET** le troisième paramètre sera vide alors qu'il contiendra les informations en cas d'utilisation de la méthode **POST**. Le corps de la réponse du serveur ne pourra pas excéder 2048 octets, ce qui permet quand même de faire pas mal de choses.

La réponse du serveur est reçue par l'événement **http_response**. Voici les 4 paramètres en œuvre :

Numéro	Paramètre	Type	Description
1	request	key	Identifiant pour la requête
2	status	integer	Code HTTP (par exemple 404, voir point 5.2.3)
3	metadata	list	Liste de constantes et attributs HTTP *
4	body	string	Corps de la requête

A noter que les serveur de **SL** créent leur propre code : 499, qui signifie qu'il y a un espace dans l'**URL**, ou que le délai est trop long (60s).

En complément **LSL** propose une fonction pour encoder les caractères spéciaux dans les **URL** : **llEscapeURL** et pour faire l'opération inverse : **llUnescapeURL**.

6. Synthèse

Nous avons maintenant tous les éléments nécessaires pour créer une application. Je vous propose quelque chose de relativement simple : un concours avec vote, quelques chose de très répandu sur **SL**. La principale difficulté dans **SL** pour gérer ce genre de manifestation est la non persistance des informations. D'autre part la gestion des données n'est pas toujours facile. En faisant appel à un serveur ce sera plus simple.

Je décline toute responsabilité quant à l'utilisation de ce code dans le jeu. Sa seule vocation est didactique et il doit encore subsister quelques bugs étant donné que je n'ai pas procédé à des tests « grandeur nature » mais seulement à des simulations limitées. D'autre part je n'ai pas poussé les fonctionnalités au-delà des nécessités de l'exemple.

6.1 Le cahier des charges

Les artistes qui exposent leurs œuvres n'en exposent qu'une. Il posent eux-mêmes leur œuvre, on leur fournit un script à insérer pour la gestion des votes. Un affichage en texte flottant doit en permanence informer sur le nombre de votes et le classement. Les votants ne peuvent voter que pour un artiste par 24 heures. Ils le font simplement en cliquant sur l'œuvre.

Le script inséré dans l'œuvre doit se connecter au serveur pour enregistrer l'artiste en vérifiant qu'il n'est pas déjà inscrit. Il faut aussi penser au reset éventuel du script. Il faut donc faire la différence entre une inscription de départ et un reset accidentel. Si on se contente de transmettre le nom ou la clé de l'artiste on ne pourra pas faire la différence. Il est d'ailleurs plus judicieux de transmettre la clé, le nom est facile à retrouver à partir de là alors que l'inverse n'est pas toujours vrai. On va donc transmettre également la clé de œuvre. Lorsque la demande arrive au serveur avec la clé de l'artiste et la clé de l'œuvre, si ces deux informations sont déjà en mémoire c'est que c'est un reset du script, on se contente de mettre à jour les informations. Dans le cas inverse on envoie un message d'erreur. Il faut aussi peut-être prendre en compte le fait que de nombreuses œuvres peuvent subir un reset de leur script simultanément avec un redémarrage de la sim. La limite d'envoi de requête concerne le nombre par objet, mais dans notre cas ce sont de multiples objets qui sont concernés. Par contre on risque de saturer un peu la sim.

Le script inséré dans l'œuvre doit aussi gérer le clic des votants en transmettant leur clé au serveur accompagnée de la clé de l'artiste. On vérifie que le votant n'a pas déjà voté dans les 24 heures. Si c'est le cas on renvoie un message d'erreur, sinon on incrémente le compteur de vote de l'artiste et on renvoie les informations à jour.

Il faut aussi prévoir un script de gestion globale qui puisse demander le classement. Un objet centralisateur affichera en permanence le classement des toris premiers en gérant les ex aequo.

6.2 La base de données

Commençons par créer la base avec **phpMyAdmin** :



Ensuite il nous faut une table pour mémoriser les informations sur les artistes :

Créer une nouvelle table sur la base **votes**

Nom: Nombre de champs:

Créons aussi les trois champs :

Serveur: localhost Base de données: votes Table: artistes

Champ	artiste	objet	votes
Type	CHAR	CHAR	INT
Taille/Valeurs ¹	36	36	
Défaut ²	aucune	aucune	Tel que défini :
Interclassement			0
Attributs			
Null	☐	☐	☐
Index	PRIMARY	UNIQUE	---
A_I	☐	☐	☐
Commentaires			

Créons la table pour les votants :

Créer une nouvelle table sur la base **votes**

Nom: Nombre de champs:

Avec ses tables :

Champ	votant	date
Type	CHAR	DATETIME
Taille/Valeurs ¹	36	
Défaut ²	aucune	aucune
Interclassement		
Attributs		
Null	☐	☐
Index	PRIMARY	---
A_I	☐	☐
Commentaires		

Notre table est prête. Passons au codage PHP.

6.3 Le code PHP

Il va nous falloir plusieurs scripts pour répondre aux différentes demandes.

6.3.1 Routines

6.3.1.1 constantes.inc.php

Il nous faut organiser notre code **PHP**. Une bonne pratique consiste à regrouper dans des fichiers distincts les éléments qui doivent être utilisés à partir de différents scripts. En particulier les renseignements concernant la connexion peuvent figurer dans un fichier particulier, il est ainsi facile de passer d'un test en local à la mise en œuvre sur serveur distant :

```
define('SERVER', 'localhost');  
define('USER', 'root');  
define('PASS', 'mysql');  
define('BASE', 'votes');
```

La fonction **define** permet de définir des constantes. Il est d'usage de mettre leurs noms en capitales. Ici on retrouve les paramètres de connexion pour un serveur local créé avec **EasyPhp**. Pour un serveur distant chez « free » par exemple le serveur s'appelle « sql.free.fr ». J'ai nommé ce fichier « constantes.inc.php ». Il est aussi classique de prévoir l'ajout de « inc » pour les fichiers inclus, ce qui n'empêche pas d'ajouter le suffixe classique « php ».

6.3.1.2 connexion.inc.php

J'ai aussi prévu un fichier distinct pour la connexion à la base que j'ai appelé « connexion.inc.php » :

```
include ("constantes.inc.php");  
// Connexion  
$link = mysql_connect(SERVER, USER, PASS);  
if (!$link)  
    die('erreur');  
// Choix de la base  
$db_selected = mysql_select_db(BASE);  
if (!$db_selected)  
    die('erreur');
```

Vous remarquez la première instruction qui prévoit l'inclusion du fichier comportant les paramètres de connexion que nous avons vu ci-dessus. Ce qui permet d'utiliser dans ce script les constantes définies dans le fichier inclus.

6.3.1.3 requete.inc.php

J'ai ensuite prévu un fichier pour les requêtes que j'ai appelé « requete.inc.php » :

```
function execQuery($sql)  
{  
    $result = mysql_query($sql);  
    if (!$result)  
        die('erreur');  
    return $result;  
}
```

```
function rowQuery($sql)
{
    return mysql_fetch_row(execQuery($sql));
}
```

Le fichier ne contient que 2 fonctions utilisateur : la première se contente d'exécuter la requête **SQL** passée en paramètre en renvoyant éventuellement une erreur. La seconde retourne la lecture du premier enregistrement (cas de la commande **SELECT**).

6.3.1.4 infos.inc.php

J'ai prévu une autre routine commune à plusieurs scripts, c'est celle qui renvoie les informations à afficher dans le texte flottant au-dessus de l'œuvre :

```
// Transmission nombre de votes + classement + nombre d'artistes
function infos($art)
{
    // Nombre de votes pour cet artiste
    $totalvotes = rowQuery("SELECT votes FROM artistes WHERE artiste = '$art'");
    // Nombre total d'artistes
    $totalartistes = rowQuery("SELECT COUNT(*) FROM artistes");
    // Nombre d'artistes classé devant
    $totaldevant = rowQuery("SELECT COUNT(*) FROM artistes WHERE votes > '$totalvotes[0]'");
    // Envoi des données
    echo $totalvotes[0] . "|" . ++$totaldevant[0], "|" . $totalartistes[0];
}
```

Une seule fonction avec quelques requêtes que nous allons examiner. La première est un **SELECT** classique où on demande le champ **votes** de la table **artistes** avec comme critère que le champ **artiste** doit être égal au contenu de la variable **\$art** qui est passé en paramètre à la fonction, ce qui nous permet donc de connaître le nombre de votes pour l'artiste ciblé. La deuxième requête comporte une fonction **SQL** : **COUNT**. Il est en effet possible de prévoir certains calculs dans une requête, ici on demande un comptage et comme il n'y a aucun critère particulier on compte tous les enregistrements de la table **artistes**, donc le nombre d'artistes. Avec la troisième requête, enfin, on compte aussi les enregistrements mais que ceux dont le nombre de votes est supérieur à celui de l'artiste concerné. Il suffit ensuite d'incrémenter ce nombre pour obtenir le classement de l'artiste. Cette procédure est indispensable dans le cas d'ex aequo. A la fin on envoie tout ça avec une fonction **echo** en séparant les valeurs par le signe « | » pour permettre à **LSL** de retrouver facilement ces valeurs.

6.3.2 initialisation.php

Lorsqu'un script **LSL** de vote est mis dans l'œuvre il envoie les informations d'initialisation : clé de l'artiste et clé de l'oeuvre. Ecrivons le script **PHP** de gestion de ces informations :

```
include ("connexion.inc.php");
include ("requete.inc.php");
include ("infos.inc.php");

$art = $_GET['artiste'];
$obj = $_GET['objet'];

// Recherche artiste et objet
```

```

$row = rowQuery("SELECT COUNT(*) FROM artistes WHERE artiste = '$art' AND objet = '$obj'");

// Cas du reset du script
if ($row[0] == 1)
    infos($art);

// Cas d'un ajout éventuel
else
{
    // Test artiste déjà présent
    $row = rowQuery("SELECT COUNT(*) FROM artistes WHERE artiste = '$art'");
    if ($row[0] == 1)
        echo "present";

    // Ajout de l'artiste
    else
    {
        execQuery("INSERT INTO artistes (artiste, objet, votes) VALUES ('$art','$obj','0')");
        echo "ajout";
    }
}
// Deconnexion
mysql_close();

```

Passé les inclusions de fichiers des routines nécessaires on récupère les valeurs de clé de l'artiste et de l'œuvre dans le tableau superglobal **\$_GET**. On lance ensuite une requête pour voir si on a déjà dans la base cet artiste avec cette œuvre. En cas de résultat positif ça veut dire qu'il y a un reset du script et qu'il faut renvoyer les informations de vote et classement avec la fonction **infos**. Sinon on vérifie que cet artiste n'a pas déjà placé une œuvre, auquel cas on renvoie le texte « present ». sinon on lance une requête d'ajout (**INSERT**). On termine en fermant la connexion pour faire propre.

Il est toujours judicieux de faire des tests de vos scripts à partir de formulaires pour vérifier que tout fonctionne correctement. Voici le formulaire que j'ai créé pour tester le script :

```

<form method="get" action="initialisation.php">
  <p>
    <label>Artiste :
    <input name="artiste" type="text">
    </label>
  </p>
  <p>
    <label>Objet :
    <input name="objet" type="text">
    </label>
  </p>
  <p>
    <label>Envoyez
    <input type="submit" name="button" value="Envoyer">
    </label>
  </p>

```

</form>

C'est du langage **HTML**. Il utilise la balise <form> pour créer un formulaire et des balises <input> pour les deux zones de saisie et le bouton d'envoi. Pour le mettre en œuvre il suffit de créer un fichier de type texte avec ces éléments et de l'enregistrer avec le suffixe « html ». Ce formulaire part du principe que votre fichier **PHP** de traitement s'appelle « initialisation.php » et qu'il se trouve dans le même répertoire que lui. Voici ce que vous obtenez :



The image shows a simple web form. It has two text input fields. The first is labeled 'Artiste : ' and the second is labeled 'Objet : '. Below these fields is a button labeled 'Envoyer'. The text 'Envoyez' is positioned to the left of the button. The entire form is enclosed in a thin black border.

Il suffit d'entrer des valeurs et de cliquer sur le bouton. Les valeurs sont transmises avec la méthode **GET**.

6.3.3 vote.php

Voyons maintenant ce qui se passe lors d'un vote. Voici le script **PHP** :

```
include ("connexion.inc.php");
include ("requete.inc.php");
include ("infos.inc.php");

$vot = $_GET['votant'];
$art = $_GET['artiste'];

// Recherche du votant
$row = rowQuery("SELECT * FROM votants WHERE votant = '$vot'");
// Date actuelle formatée pour MySql
$now = date('YmdHis');
// Votant déjà présent
if ($row != null)
{
    $timevotant = strtotime($row[1]);
    if (time() - $timevotant < 86400)
        die('erreurvote');
    // Mise à jour date du votant
    execQuery("UPDATE votants SET date = '$now' WHERE votant = '$vot'");
}
// Votant non présent, ajout
else
    execQuery("INSERT INTO votants VALUES ('$vot', '$now')");
// Nombre de votes pour cet artiste
$totalvotes = rowQuery("SELECT votes FROM artistes WHERE artiste = '$art'");
// Mise à jour votes de l'artiste'
$totalvotes[0]++;
```

```
execQuery("UPDATE artistes SET votes = '$totalvotes[0]' WHERE artiste = '$art'");
infos($art);
// Deconnexion
mysql_close();
```

Le script reçoit de **LSL** la clé du votant et celle de l'artiste. On recherche avec une première requête si le votant est déjà dans la base. Si c'est le cas on vérifie l'écoulement de 24H pour savoir s'il a le droit de voter. S'il n'a pas le droit on arrête le script avec la fonction **die** et on envoie le message « erreurvote ». Si le votant n'est pas dans la base on l'ajoute avec un **INSERT**. Ensuite on incrémente le nombre de votes de l'artiste et on envoie l'information à **LSL** avec notre fonction **infos**. Il est également judicieux de faire un formulaire de test pour ce script :

```
<form method="get" action="vote.php">
<p>
<label>Artiste :
<input name="artiste" type="text">
</label>
</p>
<p>
<label>Votant :
<input name="votant" type="text">
</label>
</p>
<p>
<label>Envoyez
<input type="submit" name="button" value="Envoyer">
</label>
</p>
</form>
```

Votre script **PHP** doit s'appeler « vote.php » et se trouver dans le même répertoire que le formulaire. Voici un visuel :



6.3.4 classement.php

Il ne nous reste plus qu'à faire le script pour le classement :

```
include ("connexion.inc.php");
include ("requete.inc.php");
```



```

// Nombre total d'artistes
$totalartistes = rowQuery("SELECT COUNT(*) FROM artistes");
if ($totalartistes [0] == 0)
    die("basevide");

// appel du classement
$result = execQuery("SELECT artiste, votes FROM artistes ORDER BY votes DESC");

$index = 0;
while ($row = mysql_fetch_assoc($result))
{
    // test pour les ex-aequo
    $resultEgaux = rowQuery("SELECT COUNT(*) FROM artistes WHERE votes = '$row[votes]'");
    for ($c = 1; $c <= $resultEgaux[0]; $c++)
    {
        if ($c > 1)
            $row = mysql_fetch_assoc($result);
        echo $row[artiste];
        if ($c == $resultEgaux[0])
            echo "|";
        else
            echo ",";
    }
    if (++$index == 3)
        break;
}
// Déconnexion
mysql_close();

```

Après avoir vérifié la présence d'artistes la requête suivante on classe les enregistrements de la table **artistes** dans l'ordre décroissant du champ **votes** (ORDER BY votes DESC). Il nous faut ensuite tester la présence éventuelle d'ex aequo. Si c'est le cas on renvoie les clé séparées avec le caractère « | ». On limite le rang de classement au troisième.

6.3.5 reset.php

Il ne nous manque plus qu'un script pour vider la base :

```

include ("connexion.inc.php");
include ("requete.inc.php");
execQuery("DELETE FROM artistes");
execQuery("DELETE FROM votants");
echo "basevide";

```

Nous avons maintenant tout notre code **PHP**.

6.4 Le code LSL

6.4.1 Script de vote

Voici le script à mettre dans les oeuvres :

```

//*****
//    Variables
//*****
key requestID;
key ava;
//*****
//    Paramètres
//*****
integer    canal = -5648521;
//*****
//    Fonctions
//*****
affichage(string s) {
    list l = llParseString2List(s, ["|"],[]);
    s = "Nombre de votes : " + llList2String(l, 0) + "\n"
        + "Classement : " + llList2String(l, 1) + "/" + llList2String(l, 2);
    llSetText(s, <1.0,1.0,1.0>, 1.0);
    llShout(canal, "RESET");
}
//*****
//    Initialisation
//*****
default {
    state_entry() {
        string URL = "http://votreserveur.fr/initialisation.php?artiste="
            + (string)llGetOwner() + "&objet=" + (string)llGetKey();
        requestID = llHTTPRequest(URL, [HTTP_METHOD, "GET"], "");
    }

    http_response(key request_id, integer status, list metadata, string body) {
        if(request_id == requestID) {
            if(body == "present")
                llOwnerSay("Vous avez droit a une seule oeuvre !");
            else if(body == "ajout") {
                llOwnerSay("Vous avez ete ajoute avec succes dans la base !");
                llShout(canal, "RESET");
                state action;
            }
            else if(body == "" || body == "erreur")
                llOwnerSay("Une erreur s'est produite, avisez l'administrateur");
            else {
                affichage(body);
                state action;
            }
        }
    }
}
//*****
//    Action
//*****

```

```

state action {
    touch_start(integer total_number) {
        ava = IIDetectedKey(0);
        string URL = "http://votreserveur.fr/votes/vote.php?votant="
            + (string)ava + "&artiste=" + (string)IIGetOwner();
        requestID = IHTTPRequest(URL, [HTTP_METHOD, "GET"], "");
    }

    http_response(key request_id, integer status, list metadata, string body) {
        if(request_id == requestID) {
            if(body == "erreurvote")
                IInstantMessage(ava, "Vous devez attendre 24 heures entre les votes !");
            else if(body == "" || body == "erreur")
                IOwnerSay("Une erreur s'est produite, avisez l'administrateur");
            else {
                affichage(body);
                IInstantMessage(ava, "Merci pour votre vote, il a ete bien pris en compte.");
            }
        }
    }
}

```

Au démarrage du script une requête est envoyée au serveur avec les clés de l'artiste et de l'objet avec la méthode **GET** et les valeurs paramétrées dans l'**URL** en faisant appel au fichier « initialisation.php » sur le serveur. Vous devez évidemment mettre à jour l'adresse pour la faire correspondre à celle de votre serveur. L'identifiant de la requête est mémorisé dans la variable **requestID**. La réponse du serveur est récupérée par l'événement **http_response**. Le message est traité et si tout se passe bien, c'est-à-dire si l'artiste a bien été ajouté dans la base, on passe dans l'état **action**. Le script est maintenant en attente des votes. Si un avatar clique sur l'objet l'événement **touch_start** est déclenché et une requête est transmise au serveur. Elle s'adresse cette fois au fichier « vote.php » avec transmission des clés de l'avatar et de l'objet. La réponse est ensuite traitée dans l'événement **http_response**. Si tout se passe bien le serveur renvoie : le nombre de votes et le classement, ces informations étant traitées par la fonction **affichage**. Le script envoie aussi une commande de mise à jour à la centrale. Celle-ci doit se trouver à moins de 100 mètres parce que j'ai utilisé la fonction **IIShout**.

6.4.2 Script de la centrale

Voici le script à mettre dans la centrale :

```

//*****
//    Variables
//*****
key requestID;
integer      index;
integer indexbis;
integer canalMenu;
integer ecouteMenu;
list noms;
list nomsbis;
string texte;
string textebis;
//*****

```

```

// Paramètres
//*****
integer canal = -5648521;
list menu = ["RESET", "UPDATE", "EXIT"];
list menuconfirmation = ["OUT", "NON"];
//*****

// Gestion de l'écoute du menu
//*****
updateEcoule(key id) {
    cancelEcoule();
    ecoleMenu = IListen(canalMenu, "", id, "");
    ISetTimerEvent(60.0);}

// -----
cancelEcoule() {
    if(ecoleMenu > 0) {
        IListenRemove(ecoleMenu);
        ecoleMenu = 0;
        ISetTimerEvent(.0);}
}
//*****

// Fonctions
//*****
integer genCanal() {
    return IFloor(IFrand(2000000.0));
}

requete(string script) {
    string URL = "http:// votreserveur.fr /votes/" + script;
    requestID = IHTTPRequest(URL, [HTTP_METHOD, "GET"], "");
}

initEgalites() {
    indexbis = 0;
    nomsbis = ICSV2List(IList2String(noms, index));
    if(ILGetListLength(nomsbis) == 1)
        IRequestAgentData((key)IList2String(noms, index), DATA_NAME);
    else
        IRequestAgentData((key)IList2String(nomsbis, indexbis), DATA_NAME);
}

integer gestionEgalites(string entete, string data) {
    if(indexbis == 0) textebis = data;
    else textebis += "," + data;
    if(++indexbis < IGetListLength(nomsbis)) {
        IRequestAgentData((key)IList2String(nomsbis, indexbis), DATA_NAME);
        return FALSE;
    }
    else {
        texte += entete + textebis;
        indexbis = 0;
        return TRUE;
    }
}
}

```

```

//*****
//    Etat unique
//*****
default
{
    state_entry() {
        IListen(canal, "", NULL_KEY, "RESET");
        ISetText("Base vide", <1.0,1.0,1.0>, 1.0);
        requete("classement.php");
    }
    listen(integer channel, string name, key id, string message) {
        if(channel == canal && message == "RESET")
            requete("classement.php");
        else if(channel == canalMenu) {
            updateEcoule(id);
            if(message == "RESET")
                IIDialog(id, "\nVraiment vider la base ?", menuconfirmation, canalMenu);
            if(message == "OUI") {
                requete("reset.php");
                ISetText("Base vide", <1.0,1.0,1.0>, 1.0);
                cancelEcoule();}
            else if(message == "UPDATE") {
                requete("classement.php");
                cancelEcoule();}
            else if(message == "EXIT")
                cancelEcoule();}
        }
    http_response(key request_id, integer status, list metadata, string body) {
        if(request_id == requestID) {
            if(body == "" || body == "erreur")
                IOwnerSay("Une erreur s'est produite, avisez l'administrateur");
            else if(body == "" || body == "basevide")
                IOwnerSay("Base mise à zéro");
            else {
                texte = "";
                noms = IParseString2List(body, ["|"],[]);
                if(IListLength(noms) == 0)
                    ISetText("Pas d'artiste", <1.0,1.0,1.0>, 1.0);
                else {
                    index = 0;
                    initEgalites();}
            }
        }
    }
    dataserver(key requested, string data) {
        if(index == 0) {
            if(IListLength(nomsbis) == 1)
                texte = "Premier : " + data;
            else
                if(!gestionEgalites("Premiers : ", data)) return;
        }
    }
}

```

```

    }
    else if(index == 1) {
        if(llGetListLength(nomsbis) == 1)
            texte += "\nSecond : " + data;
        else
            if(!gestionEgalites("\nSeconds : ", data)) return;
    }
    else if(index == 2) {
        if(llGetListLength(nomsbis) == 1)
            texte += "\nTroisième : " + data;
        else
            if(!gestionEgalites("\nTroisièmes : ", data)) return;
    }
    if(++index < llGetListLength(noms))
        initEgalites();
    else
        llSetText(texte, <1.0,1.0,1.0>, 1.0);
}
touch_start(integer total_number) {
    key ava = llDetectedKey(0);
    if(ava == llGetOwner()) {
        canalMenu = genCanal();
        updateEcoule(ava);
        llDialog(ava, "\nChoisissez une option", menu, canalMenu);}
}
timer() {
    cancelEcoule();
    llWhisper(PUBLIC_CHANNEL, "Delai écoulé, cliquez à nouveau pour obtenir le menu");}
}

```

Au démarrage du script on lance une requête de classement au cas où, on met aussi en route une écoute du canal pour recevoir le message « RESET » des scripts de vote. Il y a également un événement **touch_start** pour ouvrir un menu pour l'**Owner**. Ce menu permet de vider la base de données ou de mettre à jour le classement. Normalement cette dernière action ne doit pas être nécessaire puisque les scripts de vote envoient un message de mise à jour à chaque vote, mais des fois les messages se perdent sur SL. Un événement **listen** permet donc d'écouter les deux canaux. Si on reçoit un message « RESET » d'un script de vote on lance une requête pur récupérer le classement. Ce classement est récupéré dans l'événement **http_response**. A ce niveau il faut récupérer les noms des avatars à partir des clés transmises, ce qui ne peut se faire que par un **llRequestAgentData** (la fonction **llKey2Name** ne fonctionne que si l'avatar est dans le même simulateur). La difficulté de codage vient du fait qu'il peut y avoir des ex aequo.

7. Serveur LSL

Nous avons vu précédemment la possibilité pour un script **LSL** de lancer une requête **HTTP** à destination d'un serveur externe. Il existe désormais la possibilité inverse : un script externe à **SL** peut envoyer une requête **HTTP** à un script **LSL**, ce qui signifie que ce dernier peut fonctionner comme un serveur **HTTP**. Jusqu'à présent on ne disposait que des fonctionnalités **XML-RPC** (avec des limitations en montée de charge) pour interroger un script **LSL** de l'extérieur. On pouvait aussi interroger régulièrement un serveur pour savoir s'il avait quelque chose à nous demander, ce qui n'est pas très élégant. Nous disposons maintenant de fonctionnalités plus simples. On peut par exemple envisager la création de frontaux **WEB** à nos applications **LSL** pour le commerce ou le jeu. Mais il faut bien connaître les limites et les contraintes de ces nouvelles possibilités.

7.1 Une URL temporaire

Un serveur a besoin d'une **URL** (**U**niform **R**essource **L**ocator) pour pouvoir être appelé. Votre script **LSL** servant de serveur en a donc besoin d'une qu'il doit demander par l'intermédiaire de la fonction **llRequestURL**. Cette fonction renvoie un identifiant de type **key** qui sert à reconnaître la demande lors du retour de la réponse interceptée par le nouvel événement **http_request**. Cette précaution n'étant évidemment utile qu'en cas de mélange possible au niveau des messages reçus. Voici un script minimaliste qui permet d'obtenir une **URL** :

```
default
{
    state_entry() {llRequestURL();}
    http_request(key id, string methode, string corps)
    {
        if (methode == URL_REQUEST_GRANTED)
            llOwnerSay("URL : " + corps);
        else if (methode == URL_REQUEST_DENIED)
            llOwnerSay("URL non disponible : ");
    }
}
```

Si tout se passe bien vous obtenez une **URL** valide du genre :

<http://sim5502.agni.lindenlab.com:12046/cap/273ecd15-3563-9b18-0526-7bc734123fdb>

Mais pour combien de temps ? Vous perdez cette **URL** dans les cas suivants :

- lors d'un rez (événement **on_rez**),
- lorsque le script est réinitialisé (événement **state_entry** de l'état **default**),
- lors d'un changement de région (événement **changed** avec drapeau **CHANGED_REGION**),
- lors d'une téléportation (événement **changed** avec drapeau **CHANGED_TELEPORT**),
- lorsqu'une région redémarre (événement **changed** avec drapeau **CHANGED_REGION_RESTART**).

Vous devez tenir compte de ces contraintes lorsque vous créez un serveur. Si vous perdez votre **URL** à cause d'un des événements susmentionnés vous devez en redemander une et la communiquer aux scripts extérieurs

susceptibles d'appeler votre serveur. Vous pouvez aussi faire appel à un service externe qui permet d'avoir une **URL** permanente mais je n'ai pas encore expérimenté la validité de cette option. Toujours est-il que votre script doit intercepter les événements sensibles pour demander la nouvelle **URL**. Voici le script précédent complété pour tenir compte de ces éléments :

```
default
{
    state_entry() {llRequestURL();}
    http_request(key id, string methode, string corps)
    {
        if (methode == URL_REQUEST_GRANTED)
            llOwnerSay("URL : " + corps);
        else if (methode == URL_REQUEST_DENIED)
            llOwnerSay("URL non disponible : ");
    }
    on_rez(integer start_param) {llRequestURL();}
    changed(integer change)
    {
        if(change & (CHANGED_REGION | CHANGED_TELEPORT | CHANGED_REGION_START))
            llRequestURL();
    }
}
```

C'est évidemment un cas d'école illustratif pour bien comprendre les cas de perte de l'**URL** et les possibilités d'intercepter cette perte. Notez le nouveau drapeau **CHANGED_REGION_START** qui permet de savoir quand une région redémarre.

Si vous avez besoin d'une **URL** sécurisée il faut remplacer la fonction **llRequestURL** par **llRequestSecureURL**.

Vous pouvez libérer l'**URL** avec la fonction **llReleaseURL**.

7.2 URL disponibles

Peut-être vous posez-vous la question de savoir pourquoi il n'y aurait pas d'**URL** disponible. La principale raison est qu'il existe certaines limitations.

7.2.1 Limitations liées aux régions

La règle est la même que pour le nombre de prims mais sans tenir compte du bonus éventuel. Autrement dit si vous disposez de 200 prims sur une parcelle avec un bonus de X2 vous disposez de 100 **URL** pour les scripts de vos objets posés sur cette parcelle.

7.2.2 Limitations liées aux résidents

Chaque résident dispose de 38 **URL** pour ses attachements répartis comme il le veut. Étant donné la perte de l'**URL** en cas téléportation, de changement de région ou de rez, le fait de créer un serveur sur un attachement n'est pas vraiment de tout repos mais peut être intéressant dans certains cas.

7.2.3 Limitations des requêtes

Il existe un certain nombre de limitations au niveau des requêtes :

- la taille du corps est limitée à 2Ko,
- la taille des entêtes est limitée à 255 octets,

- le codage du contenu est en UTF8,
- la pile d'attente des requêtes a une capacité de 64, toute requête au-delà de cette limite est perdue.

7.3 Réponses aux requêtes

Lorsque votre serveur reçoit une requête vous l'interceptez dans l'événement **http_request**, Cet événement comporte 3 paramètres :

1	id	key	Identifiant unique de la requête
2	methode	string	Méthode GET , POST , PUT ou DELETE
3	body	string	Corps de la requête

Votre serveur doit répondre (sinon à quoi ça sert ?). Pour cela il utilise la fonction **IIHTTPResponse**. Cette fonction attend aussi 3 paramètres :

1	id	key	Identifiant reçu dans l'événement http_request correspondant
2	statut	integer	Statut HTTP (200, 400, 404, etc...)
3	body	string	Corps de la requête

7.4 Un exemple simple

Considérons un cas simple. Vous avez un objet dont vous voulez connaître la position à partir d'un serveur externe. Vous créez un script dans votre objet qui fonctionne en serveur :

```
string URL = "http://votreserveur.fr/initialisation.php?url=";
float DELAI_SEND_URL = 360.0;
float DELAI_REQUEST_URL = 60.0;

string memoURL;
key requestID;

sendURL()
{
    requestID = IIHTTPRequest(memoURL, [HTTP_METHOD, "GET"], "");
    IISOwnerSay("Envoi de l'url : " + memoURL);
    IISetTimerEvent(DELAIS_SEND_URL);
}

default
{
    state_entry() {IIRequestURL();}
    http_request(key id, string methode, string corps) {
        if (methode == URL_REQUEST_GRANTED) {
            memoURL = URL + corps;
            sendURL();
        }
        else if (methode == URL_REQUEST_DENIED) {
            IISOwnerSay("Pas d'URL ! Je redemande dans " + (string)DELAIS_REQUEST_URL + "
secondes");
            IISleep(DELAIS_REQUEST_URL);
            IIRequestURL();
        }
    }
}
```

```

    }
    else if (methode == "GET")
        llHTTPResponse(id, 200, "Ma position est : " + llGetRegionName() + " : " +
(string)llGetPos());
    }
    on_rez(integer start_param) {llRequestURL();}
    changed(integer change) {
        if(change & (CHANGED_REGION | CHANGED_TELEPORT |
CHANGED_REGION_START))
            llRequestURL();
    }
    http_response(key request_id, integer status, list metadata, string body) {
        if(request_id == requestID && body == "URL_OK") {
            llOwnerSay("URL reçue par le serveur");
            llSetTimerEvent(.0);
        }
    }
    timer() {sendURL();}
}

```

En réponse à une requête de type “GET” le serveur renvoie la valeur de la position de l'objet qui contient le script. D'autre part en cas de changement de la valeur de l'URL celle-ci est envoyée à un serveur externe pour mise à jour. Ainsi la continuité de fonctionnement du système est assurée. Cela implique au niveau du serveur externe une mémorisation de l'information. Dans le cas d'un serveur classique **PHP/MySQL** il est alors nécessaire de disposer d'une table pour mémoriser cette information. J'ai donc créé une table **urls** avec un seul champ **url** de type **VARCHAR** de longueur 250.

Côté **PHP** j'ai repris les deux scripts de configuration **constantes.inc.php** :

```

define('SERVER', 'localhost');
define('USER', 'root');
define('PASS', 'mysql');
define('BASE', 'urls');

```

Evidemment les valeurs ci-dessus doivent être adaptées en fonction du serveur que vous utilisez. Le serveur que j'ai utilisé pour mes tests est celui de free : **sql.free.fr**.

J'ai repris aussi **connexion.inc.php** :

```

include ("constantes.inc.php");
// Connexion
$link = mysql_connect(SERVER, USER, PASS);
if (!$link)
    die('erreur');
// Choix de la base
$db_selected = mysql_select_db(BASE);
if (!$db_selected)
    die('erreur');

```

Puisque nous devons à nouveau faire appel à une base **MySQL**.

J'ai ensuite créé un script **initialisation.php** destiné à mettre à jour la valeur de l'**URL** dans la base :

```
include ("connexion.inc.php");
function query($sql)
{
    $result = mysql_query($sql);
    if (!$result)
        die();
    return;
}
$url = $_GET['url'];
query("DELETE FROM urls");
query("INSERT INTO urls VALUES ('$url')");
echo "URL_OK";
```

Il ne me restait plus qu'à créer le script d'interrogation :

```
include ("connexion.inc.php");
function select($sql)
{
    $result = mysql_query($sql);
    if (!$result)
        die('erreur');
    return $result;
}
$result = select("SELECT * FROM urls");
$row = mysql_fetch_row($result);
echo "<a href=\"\$row[0]\">Demander la position</a> ";
```

En faisant un petit test chez moi j'obtiens ça dans mon navigateur préféré (**Firefox** pour ne pas le nommer) :

```
Ma position est : Nostra Terza Vita : <15.73811, 220.60420, 500.55070>
```

Il ne s'agit évidemment que d'un exemple sommaire mais fonctionnel qui devrait vous donner envie de créer des applications plus ambitieuses en contrôlant par exemple un grand nombre d'objets et une interface **HTML** soignée et tous les délires qui peuvent germer de votre imagination !